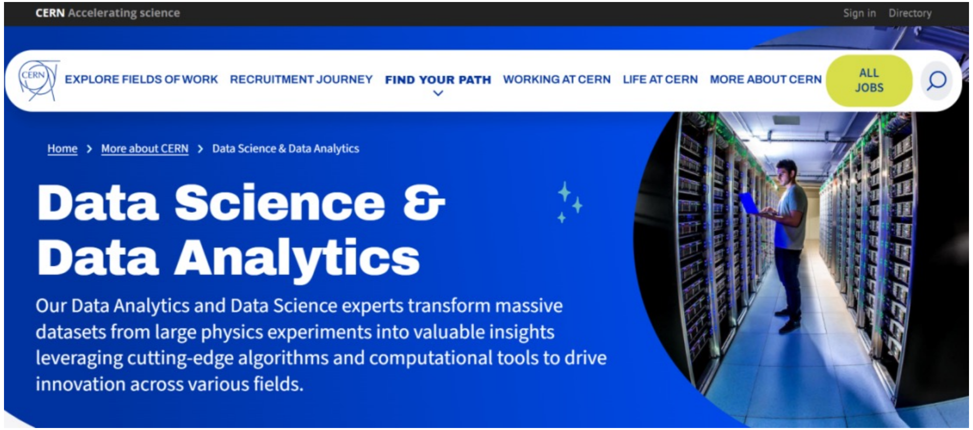
Data science and data exploration

Dr. Leonid Serkin

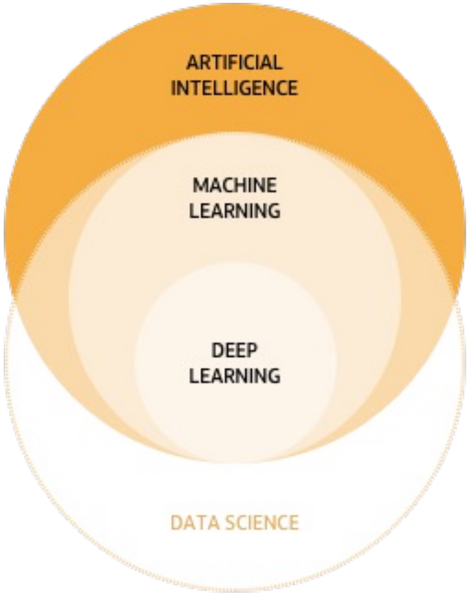
Faculty of Sciences
National Autonomous University of Mexico (UNAM)

What is data science?

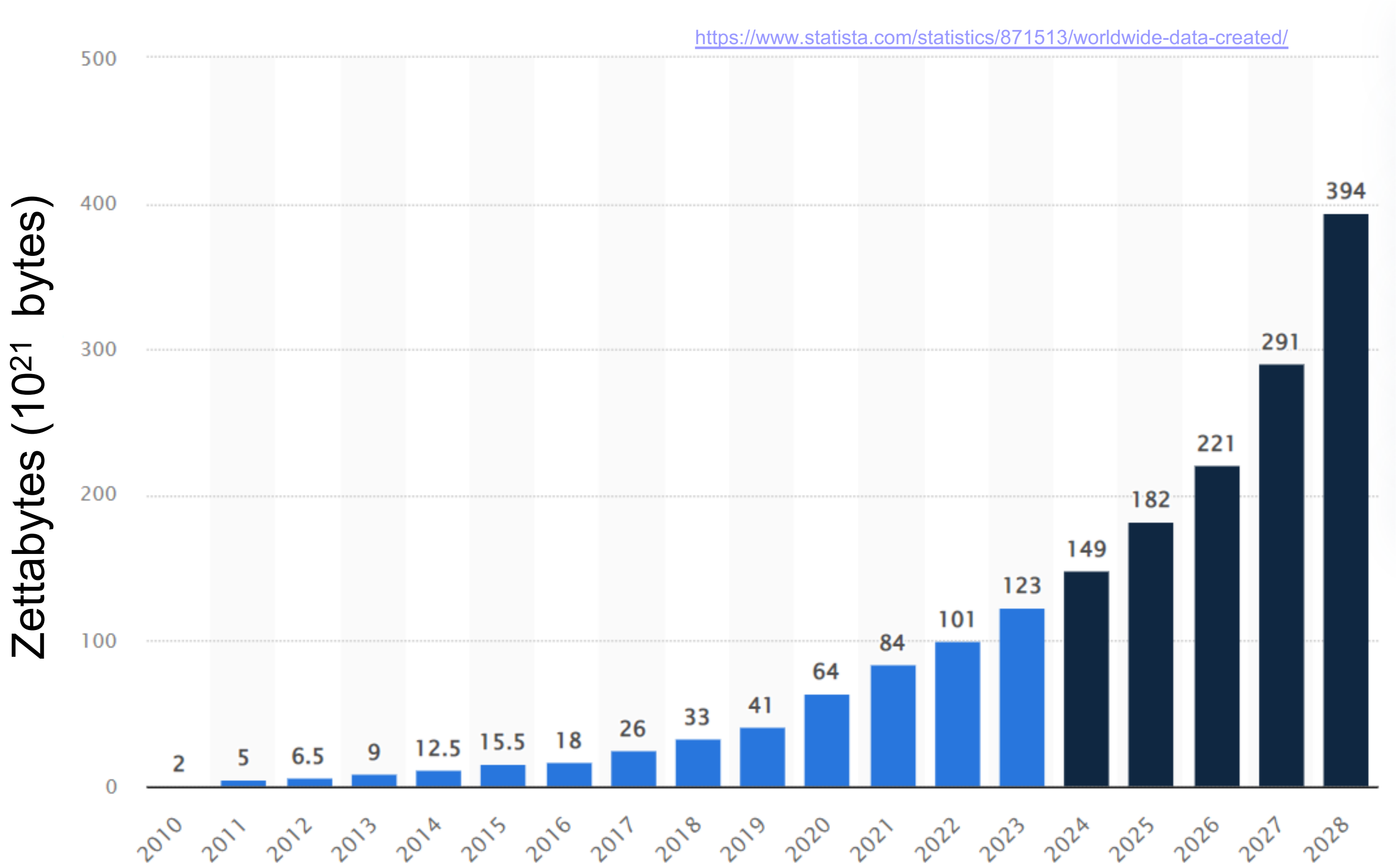
Data science is an interdisciplinary academic field that uses statistics, scientific computing, scientific methods, processing, scientific visualization and algorithms to extract or extrapolate knowledge from potentially noisy, structured, or unstructured data.



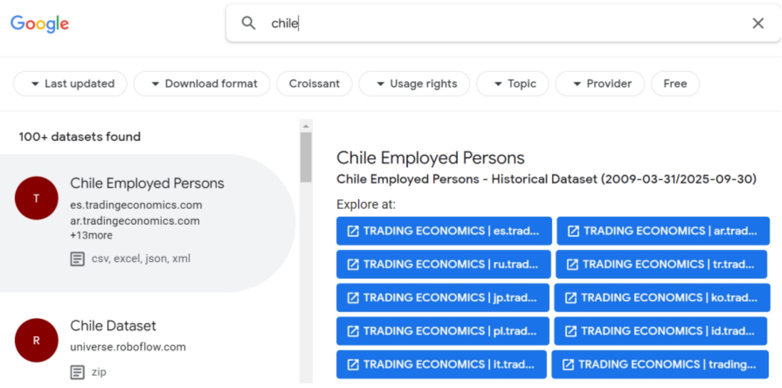
https://careers.cern/explore-careers/data-science-data-analytics



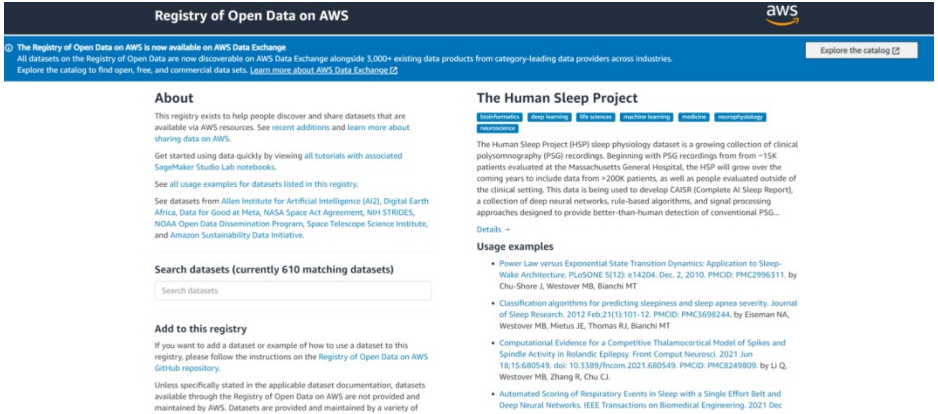
Volume of data created and consumed worldwide



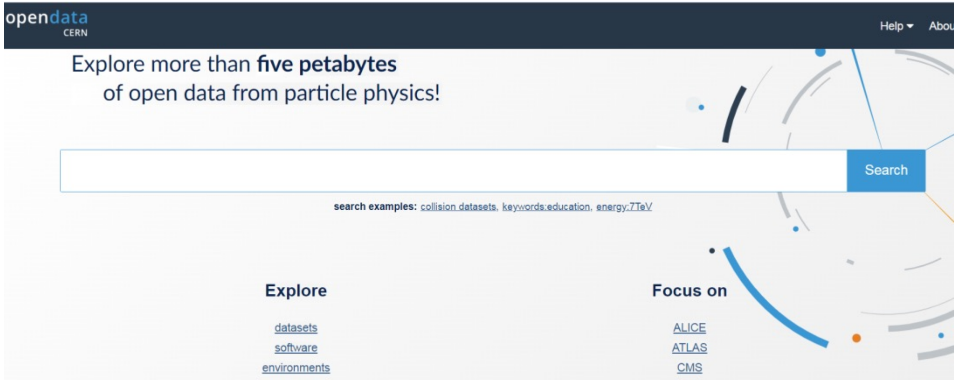
Open research data



https://datasetsearch.research.google.com

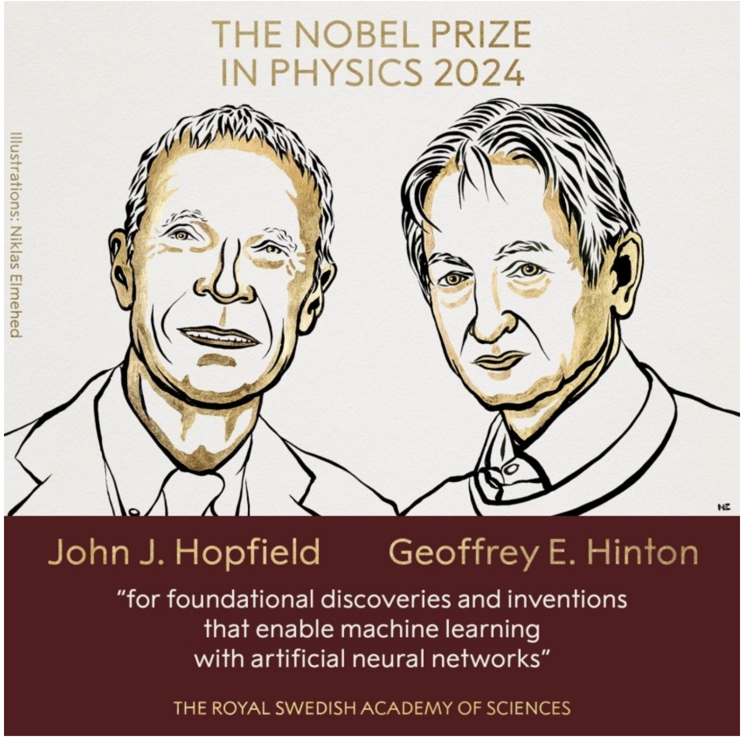


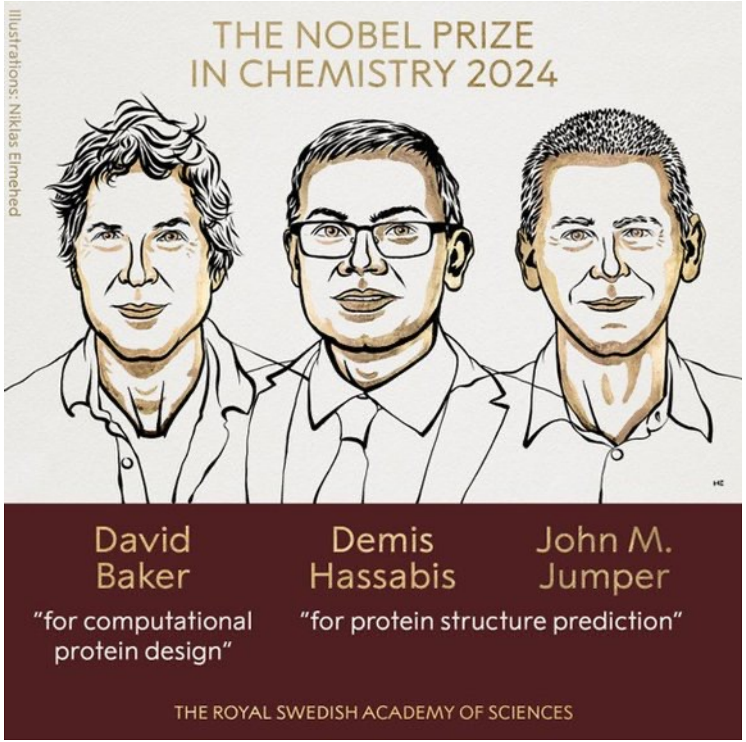
https://registry.opendata.aws/



https://opendata.cern.ch/

These are really good times to do DS and ML



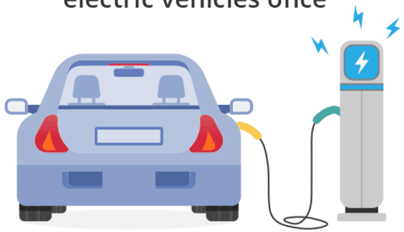


Yet, uncle Ben said:

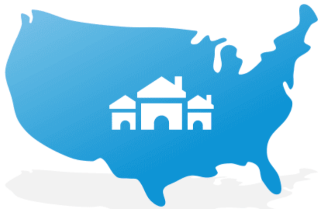
Remember, each time you ask ChatGPT for an answer

The annual amount of electricity ChatGPT uses to respond to prompts (226.82 million kWh) would be enough to:

Fully charge 3.13M
electric vehicles once



Power 21,602
U.S. households for a year

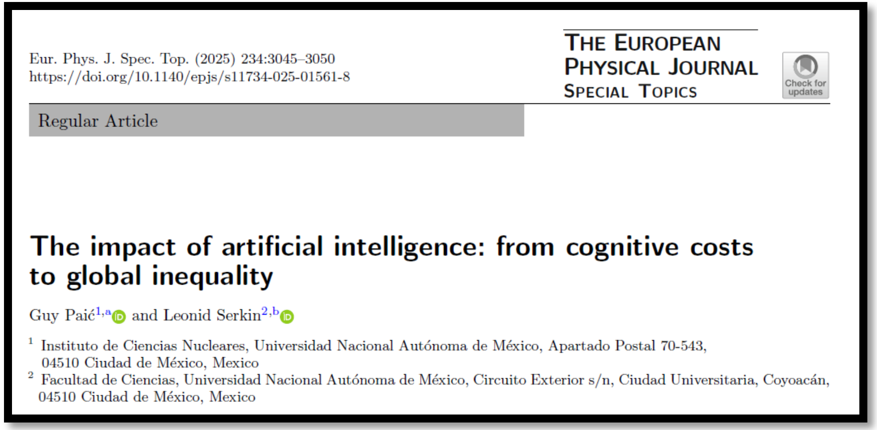


Charge 47.87 million
iPhones daily for an entire year

Supply electricity to Gibraltar for
1 year and 49 days

Data Sources: U.S. Energy Information Administration, Electric Power Research Institute (EPRI), EV Database, U.S. Department of Energy (DOE), Wikipedia

BestBrokers.com



Eur. Phys. J. Spec. Top. (2025) 234:3045–3050
https://doi.org/10.1140/epjs/s11734-025-01561-8

THE EUROPEAN PHYSICAL JOURNAL SPECIAL TOPICS

Check for updates

Regular Article

The impact of artificial intelligence: from cognitive costs to global inequality

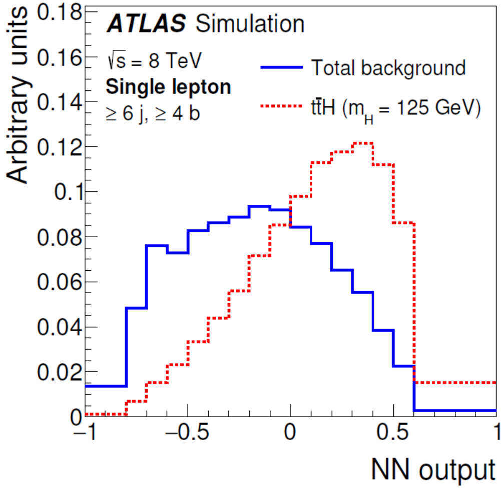
Guy Paić[1,a] and Leonid Serkin[2,b]

[1] Instituto de Ciencias Nucleares, Universidad Nacional Autónoma de México, Apartado Postal 70-543, 04510 Ciudad de México, Mexico
[2] Facultad de Ciencias, Universidad Nacional Autónoma de México, Circuito Exterior s/n, Ciudad Universitaria, Coyoacán, 04510 Ciudad de México, Mexico

https://link.springer.com/article/10.1140/epjs/s11734-025-01561-8

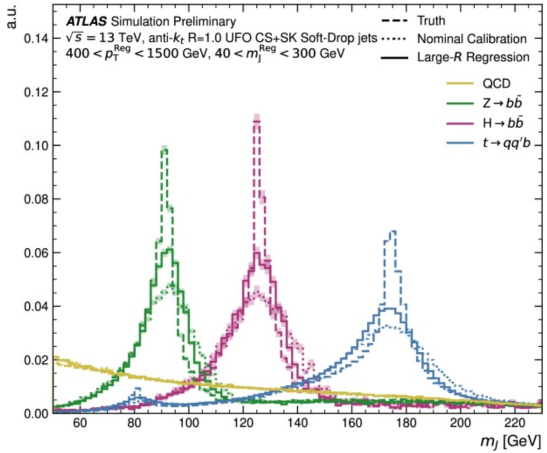
Core concepts of data science applied at CERN

CLASSIFICATION



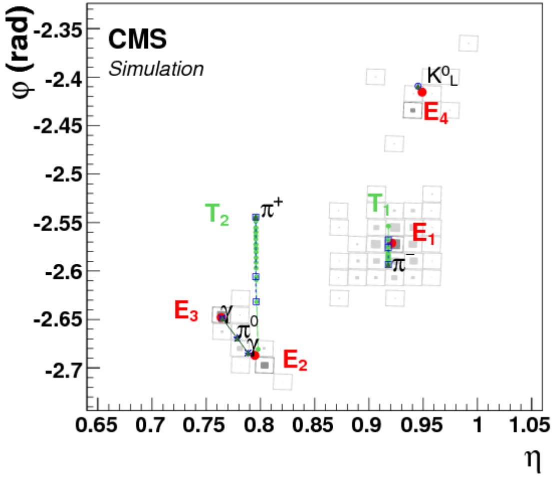
Identify particle types,
event categories,
signal vs background…

REGRESSION



Predict continuous quantities:
energies, momenta,
calibrations…

CLUSTERING



Build physics objects from
detector hits:
jets, topo-clusters, tracks…

Data science workflow: problem

1) Start by clearly defining the **scientific or technical problem** you aim to solve within the context of the CERN experiment.

Data science workflow: problem → plan

1) Start by clearly defining the **scientific or technical problem** you aim to solve within the context of the CERN experiment.

2) Outline the **planning stage**, including what measurements are needed, how data will be collected, and the experimental or computational design behind it.

Data science workflow: problem → plan → data

1) Start by clearly defining the **scientific or technical problem** you aim to solve within the context of the CERN experiment.

2) Outline the **planning stage**, including what measurements are needed, how data will be collected, and the experimental or computational design behind it.

3) Describe the process of **collecting, cleaning, and managing CERN data**, such as detector calibration, event filtering, and handling large-scale datasets

Data science workflow: problem → plan → data → analysis

1) Start by clearly defining the **scientific or technical problem** you aim to solve within the context of the CERN experiment.

2) Outline the **planning stage**, including what measurements are needed, how data will be collected, and the experimental or computational design behind it.

3) Describe the process of **collecting, cleaning, and managing CERN data**, such as detector calibration, event filtering, and handling large-scale datasets

4) Present how you **explore and analyse the data**, looking for patterns in collision events, generating hypotheses, and applying statistical or machine-learning methods.

Data science workflow: problem → plan → data → analysis → conclusion

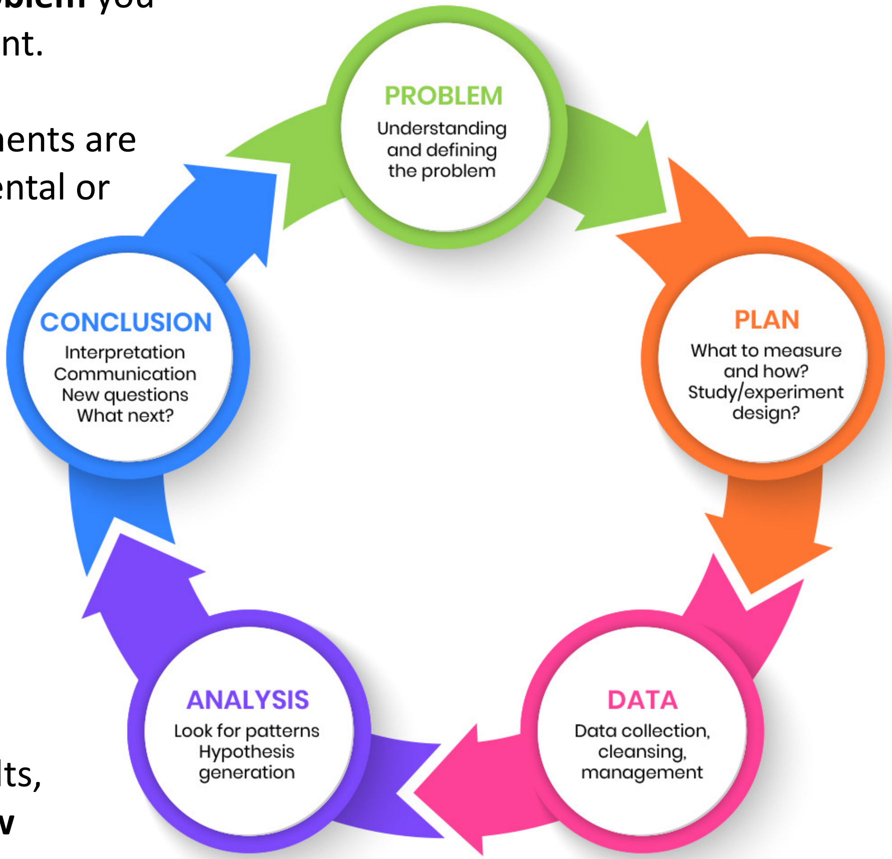
1) Start by clearly defining the **scientific or technical problem** you aim to solve within the context of the CERN experiment.

2) Outline the **planning stage**, including what measurements are needed, how data will be collected, and the experimental or computational design behind it.

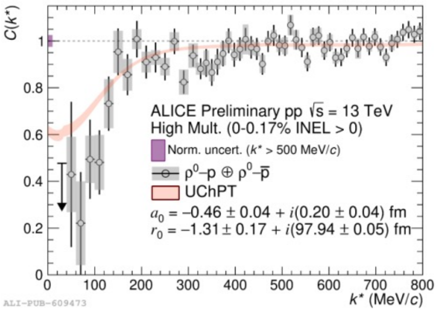
3) Describe the process of **collecting, cleaning, and managing CERN data**, such as detector calibration, event filtering, and handling large-scale datasets

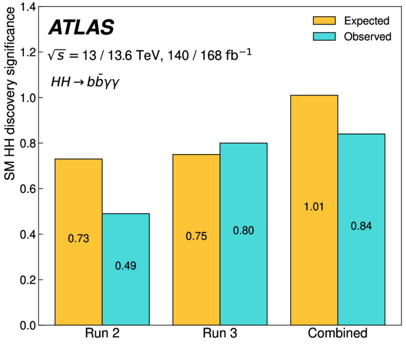
4) Present how you **explore and analyse the data**, looking for patterns in collision events, generating hypotheses, and applying statistical or machine-learning methods.

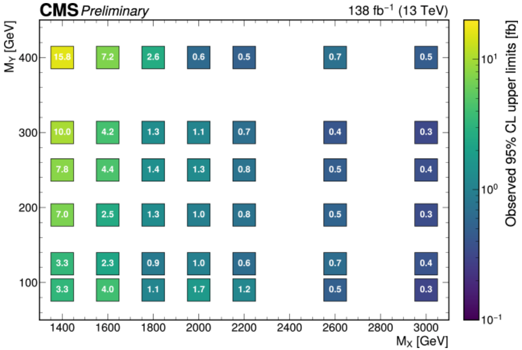
5) Summarise how you form conclusions, **interpret** results, **communicate** them to collaborators, and **identify new questions** or improvements for the future.

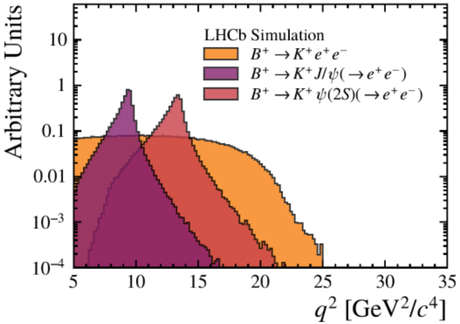


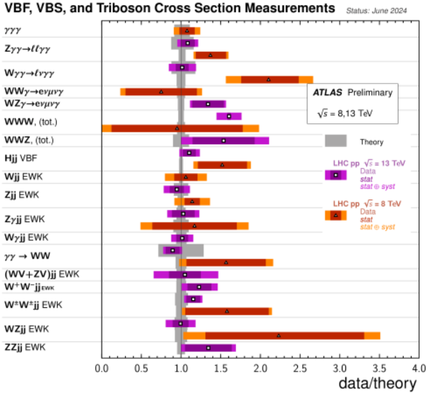
At CERN we love making beautiful data visualizations

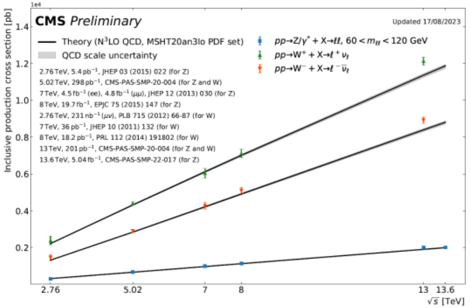


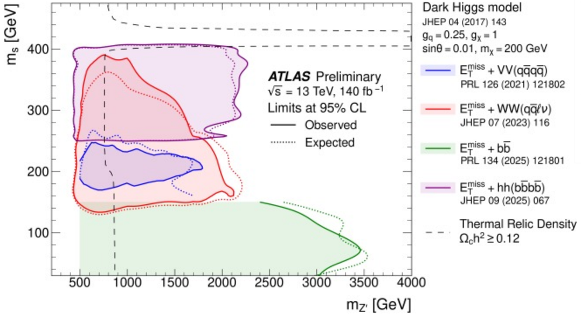


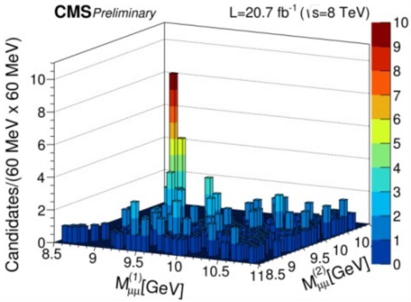




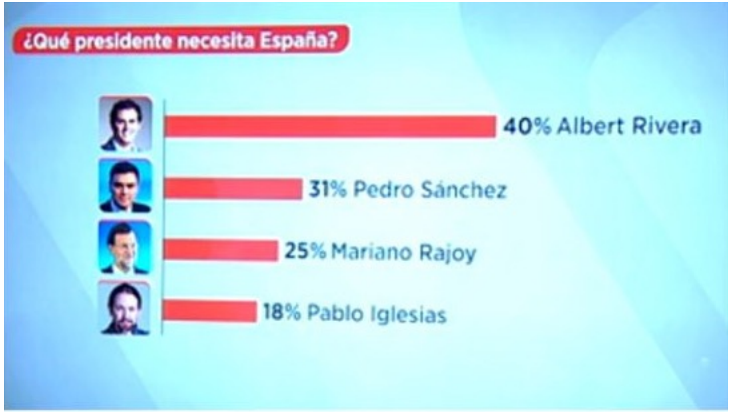


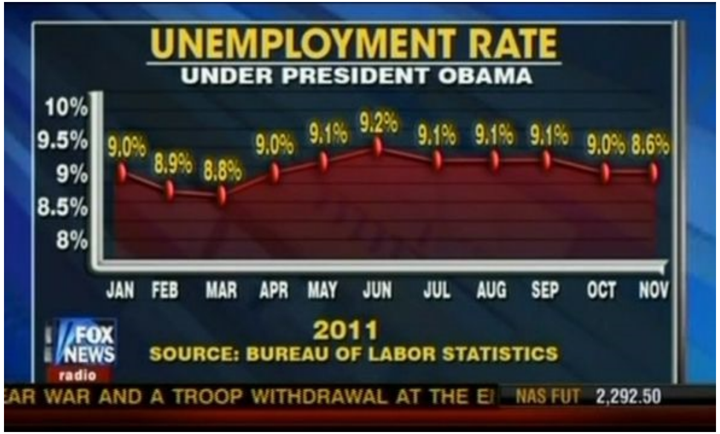






Others do as well… but they are “a bit” misleading





Our goals for today: a very interactive lecture!

- Learn an **essential skill**: apply data exploration to clearly and effectively communicate data-driven findings.

- Gain **practical tools** for the scientific analysis, processing, and interpretation of diverse types of datasets.

- Understand the **limitations** of several commonly used plots and learn why some should be avoided.

- Develop exploratory data analysis skills by investigating the data through **visualization**.

- Identify **biases** and systematic effects: detect mistakes, anomalies, and unexpected behaviour within datasets.

- Use a **modern scientific Python stack**: NumPy, Pandas, Matplotlib, Seaborn, Plotly and Scikit-learn.

- **Core techniques developed in 10 Jupyter Notebooks**: data preprocessing, summary statistics, visual data exploration, color maps, dimensionality reduction, handling missing and imbalanced data, bootstrapping, clustering, and visualising textual data.

READY TO CODE?
FOLLOW THE README AT:

https://github.com/lserkin/csc2026-lecture

Pre-processing: start simple

- **Notebook to try: Preprocessing.ipynb**
- Let's start with: `data = pd.read_csv('dataset.csv')`
- Check dataset size: **data.shape**
- Preview the first rows: **data.head()**
- General information about the dataset: **data.info()**
- How many missing values each column has: **data.isnull().sum()**
- Descriptive statistics: **data.describe()**

	target	var1	var2	var3	var4
count	10000.00000	10000.000000	10000.000000	10000.000000	10000.000000
mean	0.31460	1.041957	1678.734830	1677.217633	2.975683
std	0.46438	0.884508	10391.950014	10392.194814	1.157038
min	0.00000	-0.603788	-9782.758399	-9782.758399	0.777764
25%	0.00000	0.319062	2.114084	0.398036	2.147795
50%	0.00000	0.917863	2.158831	0.608327	2.618077
75%	1.00000	1.660586	2.246108	0.875455	3.537568
max	1.00000	3.341215	99959.752415	99959.752415	6.227424

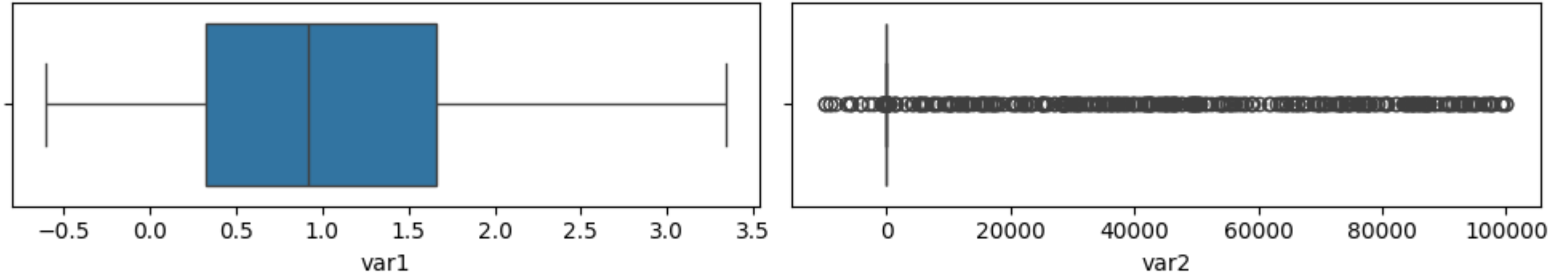
```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 17 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   target  10000 non-null  int64
 1   var1    10000 non-null  float64
 2   var2    10000 non-null  float64
 3   var3    10000 non-null  float64
 4   var4    10000 non-null  float64
 5   var5    10000 non-null  float64
 6   var6    10000 non-null  float64
 7   var7    10000 non-null  float64
 8   var8    10000 non-null  float64
 9   var9    10000 non-null  float64
 10  var10   10000 non-null  float64
 11  var11   10000 non-null  float64
 12  var12   10000 non-null  float64
 13  var13   10000 non-null  float64
 14  var14   10000 non-null  float64
 15  var15   10000 non-null  float64
 16  var16   10000 non-null  float64
dtypes: float64(16), int64(1)
memory usage: 1.3 MB
```

- In Var2 and Var3 we have **min** or **max** values that make no sense and very large **std. dev.**

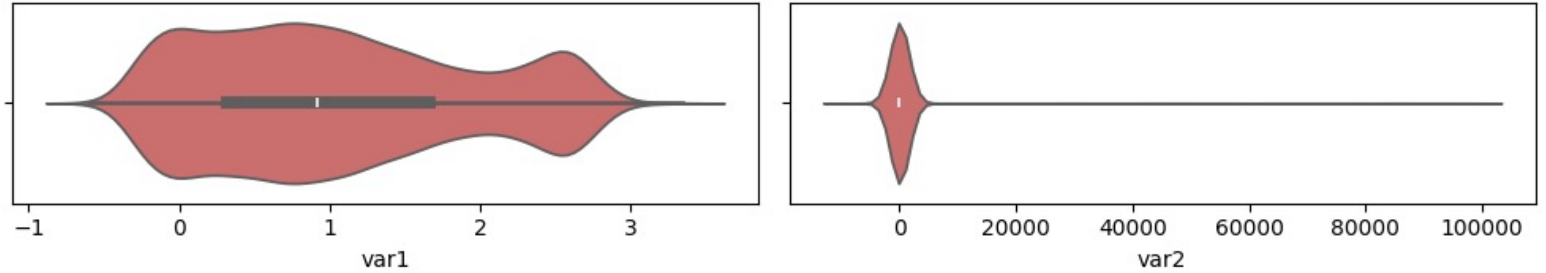
Pre-processing: outliers

- Let's find those **outliers** using Matplotlib and seaborn libraries:

- A **boxplot** shows where the middle 50% of the data lies (the box), the median (the line), and how far the non-outlier values extend (the whiskers). The bottom of the box is Q1 (25th percentile) and the top is Q3 (75th pct.)



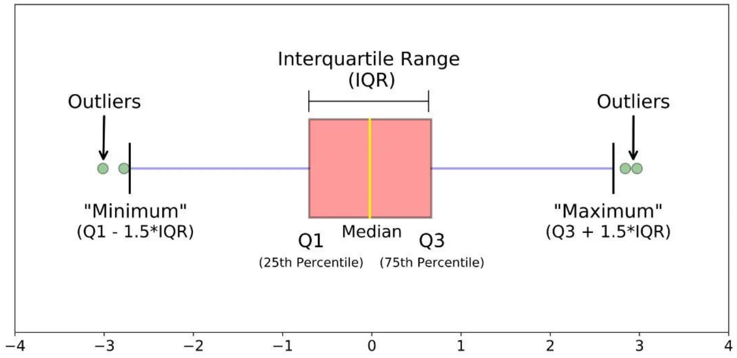
- **A violin plot** is a representation of data using kernel density estimation (KDE). Wider regions correspond to higher probability density, while narrower regions indicate sparse data. A central marker shows the median, and the overall shape provides insight into skewness and extreme values.



Pre-processing: capping outliers

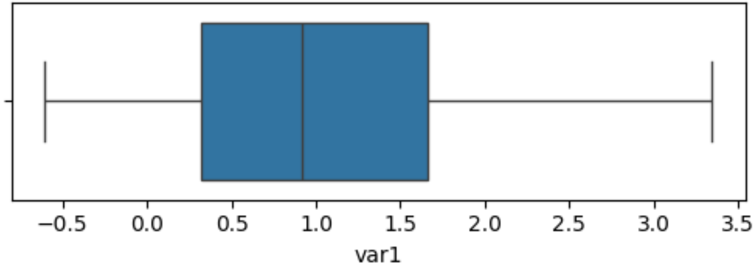
- **What can you do with outliers?**

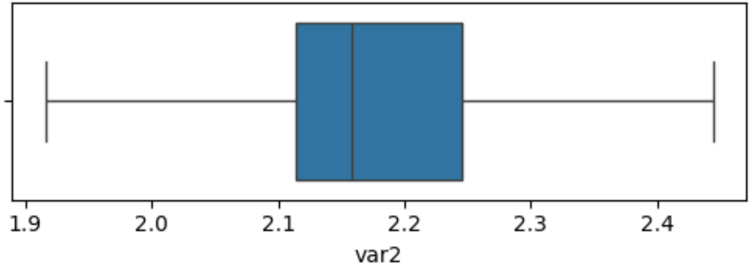
  a) Remove outliers entirely (in case they are true errors)
  b) Leave them untouched (if they are legitimate and your models are robust)
  c) Apply IQR capping (you want to reduce influence of extreme values, but hide anomalies)



- Interquartile Range (IQR) capping **limits all values** to the interval (Q1 - 1.5*IQR) to (Q3 + 1.5*IQR)
- Any value below the lower or above the upper bounds are **capped**, reducing the influence of extreme outliers:

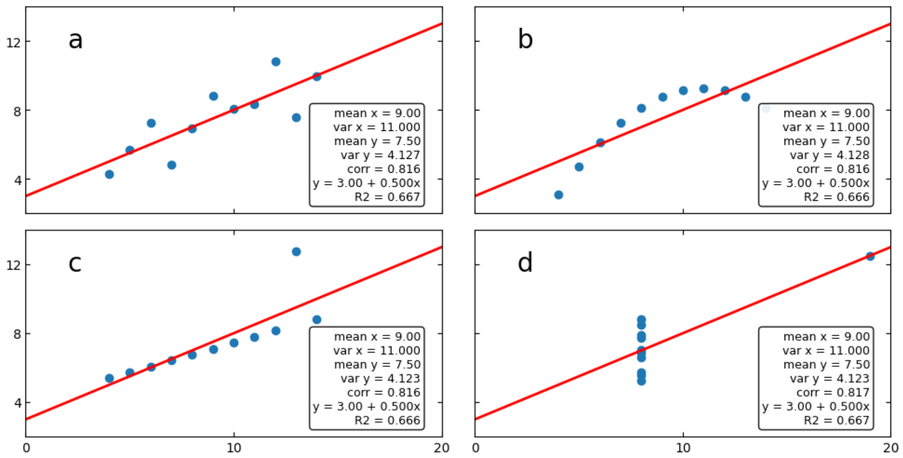
```
'var1': 0 outliers capped    | Bounds: [-1.69, 3.67]
'var2': 1203 outliers capped | Bounds: [ 1.92, 2.44]
```

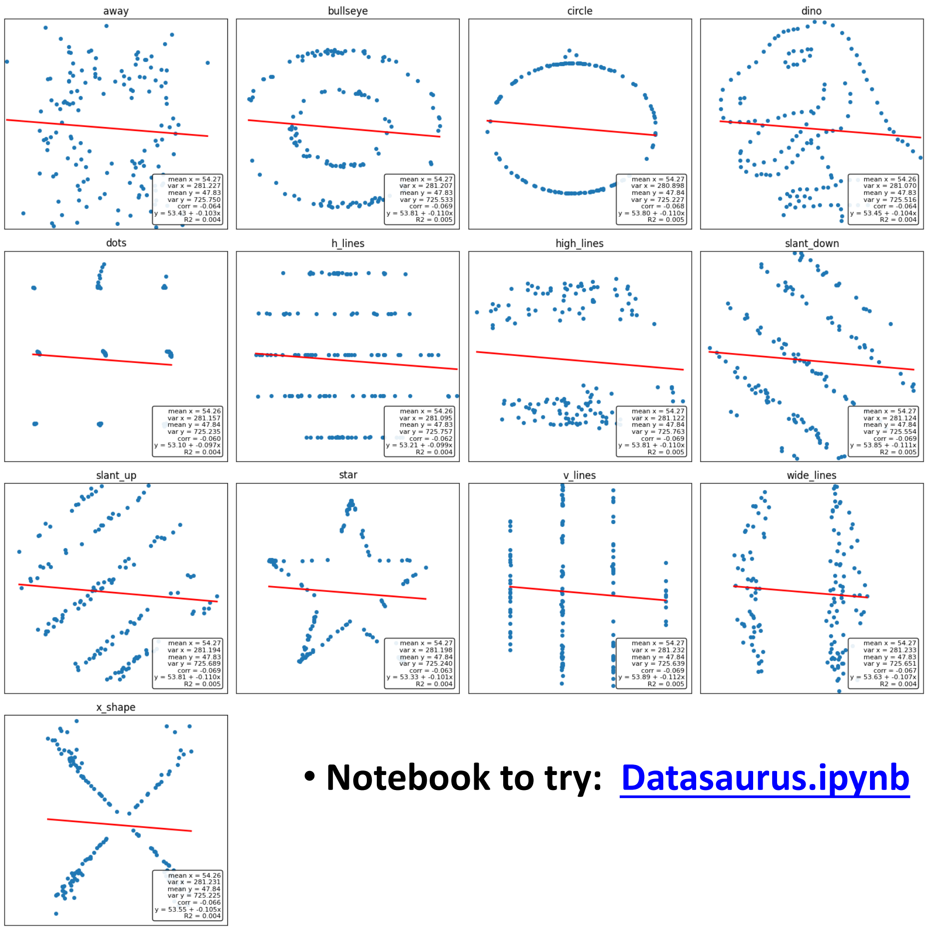




Summary statistics are not enough!

- **Anscombe's quartet**: four datasets with just 11 points that have nearly identical simple descriptive statistics, yet have very different distributions.





- Data can have **identical** summary statistics, data can have **identical** correlation values, but hide completely **different** structures.

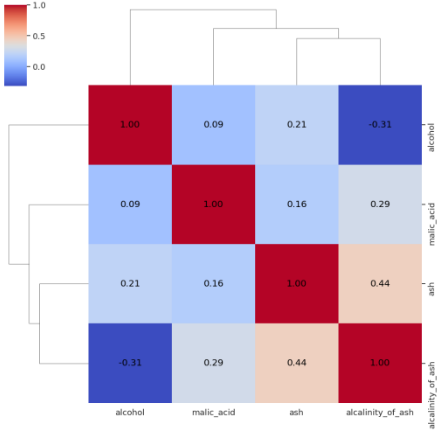
- **Always plot your data!**

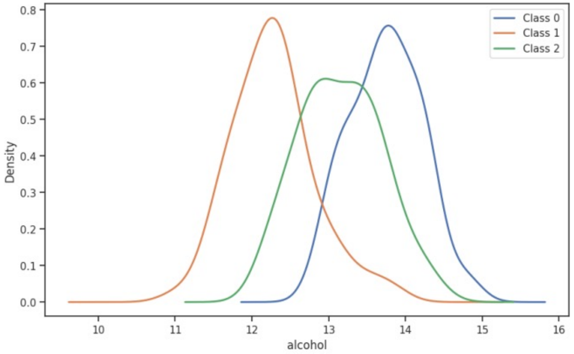
- **Notebook to try: Datasaurus.ipynb**

Visual data exploration

- **Before** building any machine learning model, **always do an EDA** (Exploratory Data Analysis): a step-by-step process used to understand, clean, visualize, and summarize your dataset

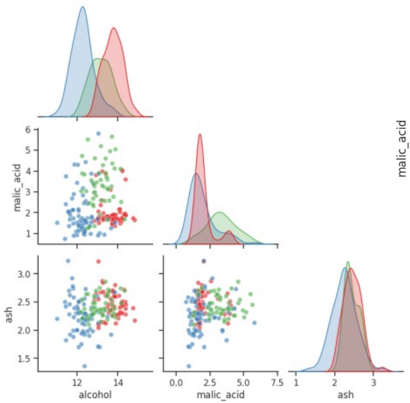
1) Start with basic dataset exploration
2) Histograms of the variables (features)
3) Kernel density (KDE) curves
4) Overlaid histograms for diffent classes
5) 2D scatter plots and matrix
6) Pearson & Spearman correlations
7) Pairplot and heatmaps

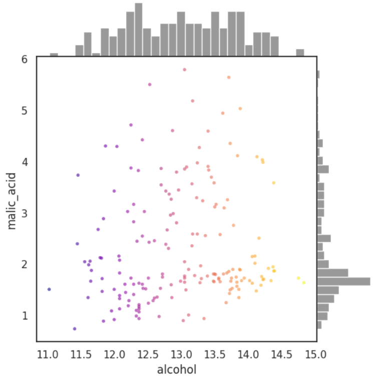




- **Notebook to try: Wine.ipynb**

- The dataset contains 13 components from a **chemical analysis of wines** grown in the same region of Italy but produced from three different types of grapes: Barolo, Barbera, and Grignolino, all red wines.

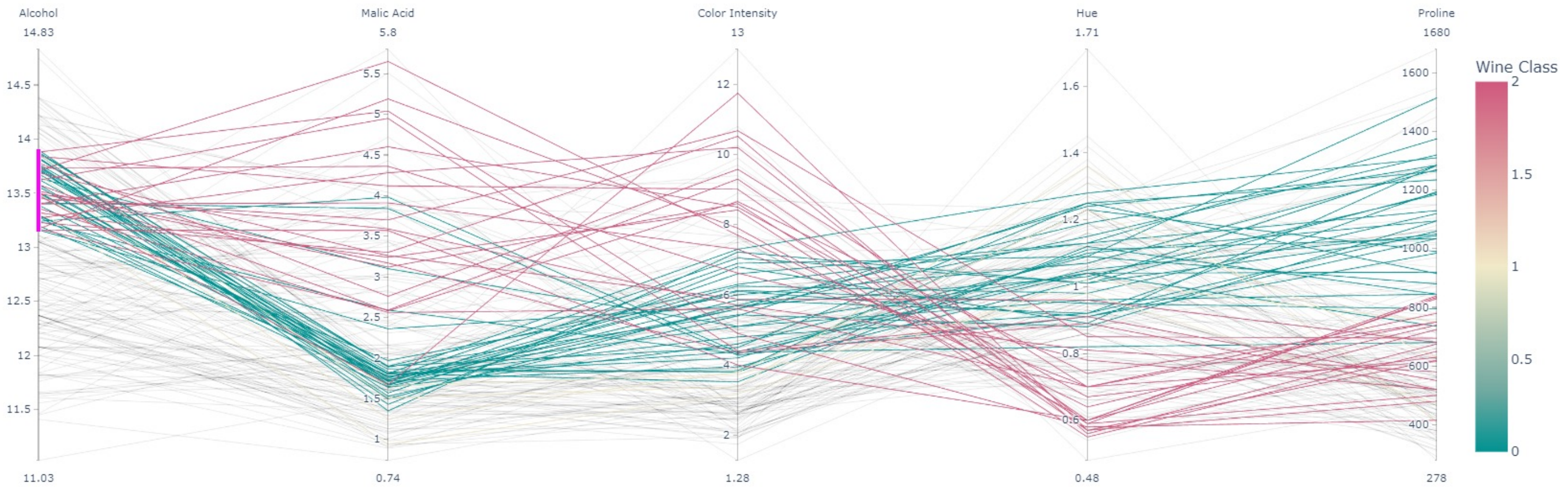




Interactive data exploration

- **Parallel coordinates plot (from plotly)**: a way to visualize multivariate data in a single 2D plot

- Shows class separability across multiple features, and supports **interactive brushing** to zoom into regions.



Correlations

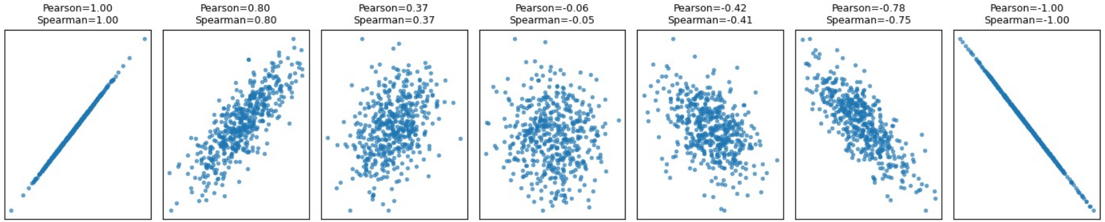
Pearson correlation df.corr()

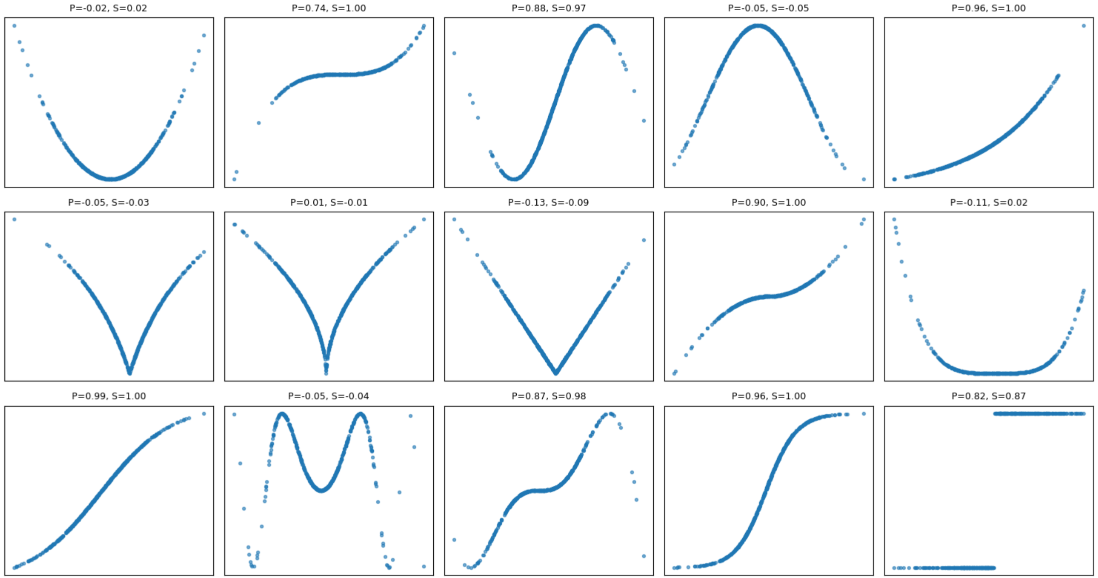
$$r_{xy} = \frac{\sum (X_i - \overline{X})(Y_i - \overline{Y})}{\sqrt{\sum (X_i - \overline{X})^2 \sum (Y_i - \overline{Y})^2}}$$

Spearman correlation df.corr(method="spearman")

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

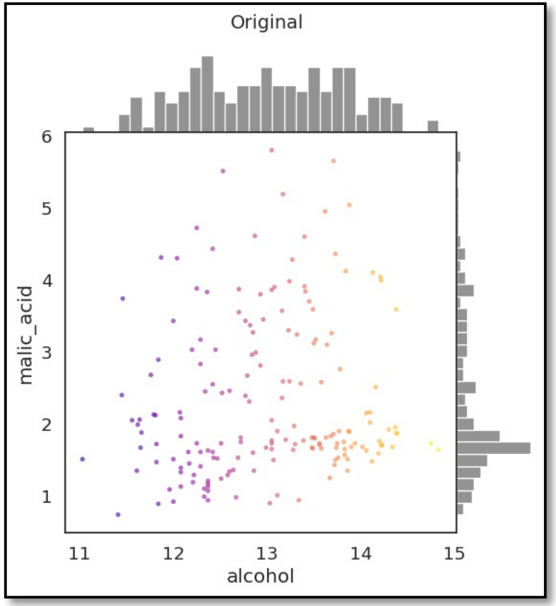
- Pearson correlation measures the strength of a **linear relationship** between variables, while Spearman correlation measures the **strength of a monotonic relationship** based on ranked values.

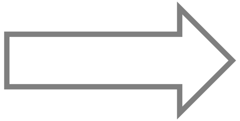


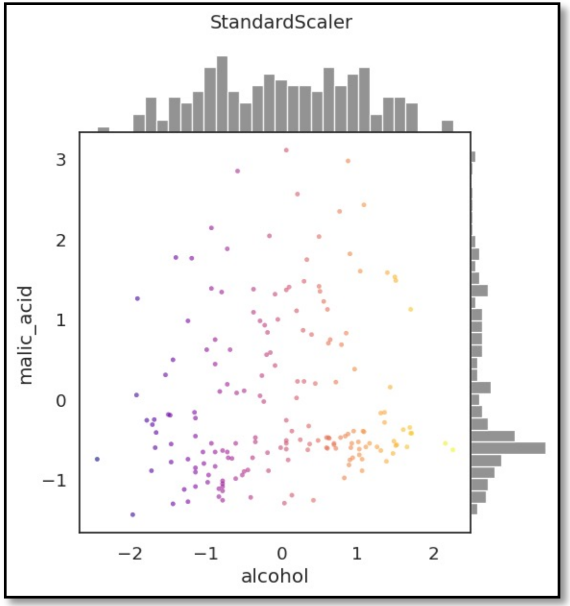


Scalers, transformers and normalizers

- Different variables have **different scales** and may contain **outliers**
- Difficulties when visualizing the data and, more importantly, can degrade the predictive performance of many ML algorithms (these might implicitly assume that features lie close to zero or that all features vary on comparable scales).
- Scalers are linear transformations: **StandardScaler produces zero-mean unit-variance features**

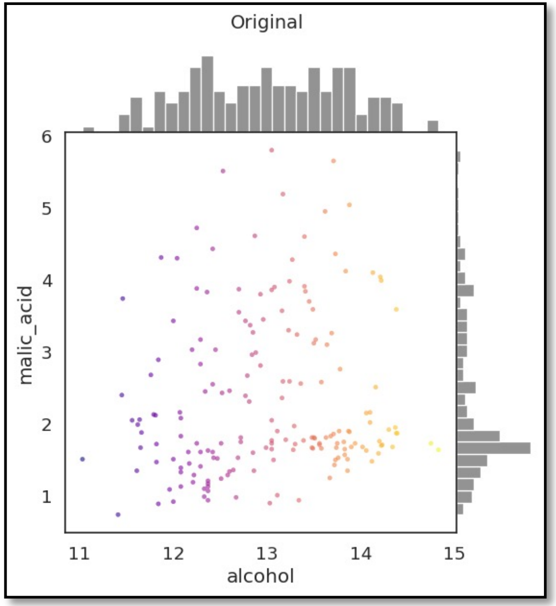


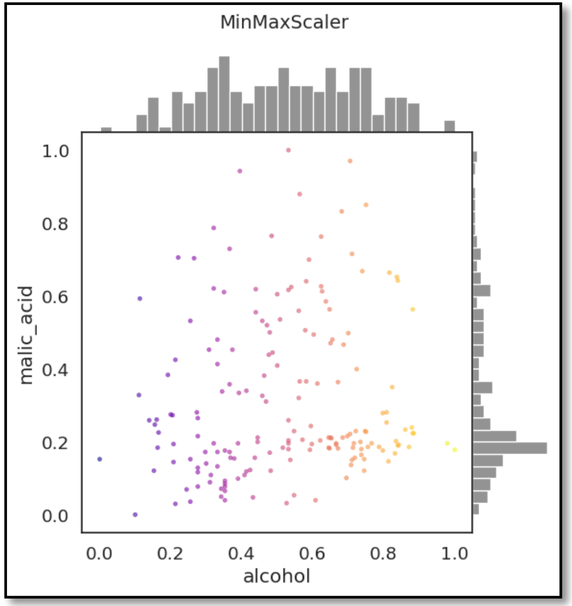




Scalers, transformers and normalizers

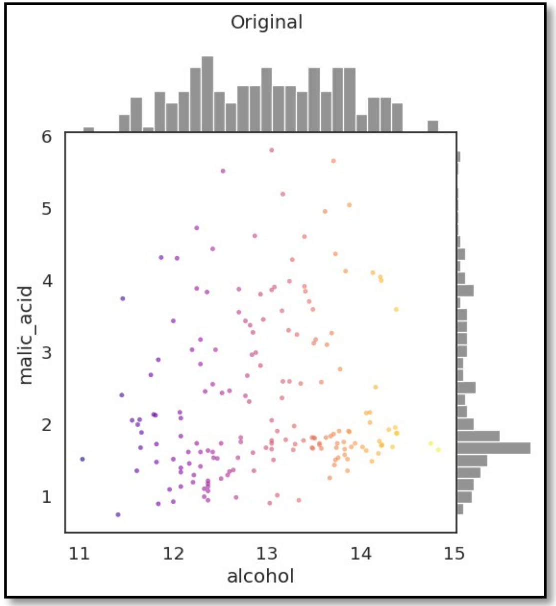
- Different variables have **different scales** and may contain **outliers**
- Difficulties when visualizing the data and, more importantly, can degrade the predictive performance of many ML algorithms (these might implicitly assume that features lie close to zero or that all features vary on comparable scales).
- Scalers are linear transformations: **MinMaxScaler rescales each feature to [0,1]**

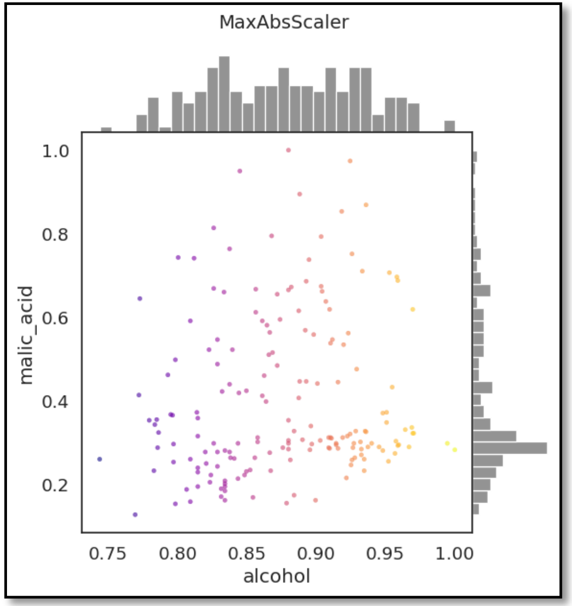




Scalers, transformers and normalizers

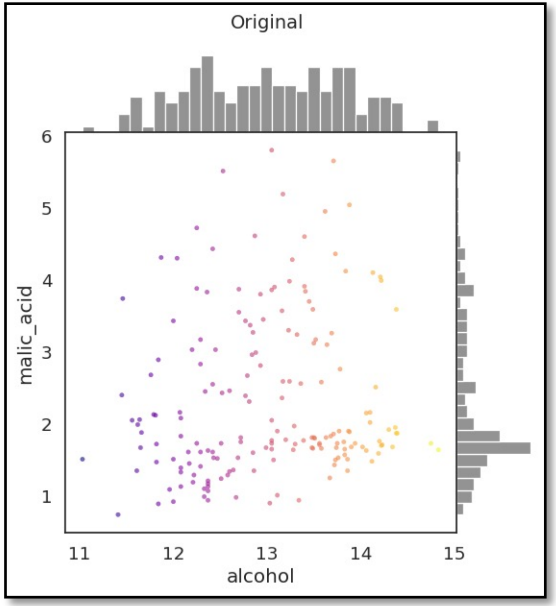
- Different variables have **different scales** and may contain **outliers**
- Difficulties when visualizing the data and, more importantly, can degrade the predictive performance of many ML algorithms (these might implicitly assume that features lie close to zero or that all features vary on comparable scales).
- Scalers are linear transformations: **MaxAbsScaler rescales to [−1,1]**

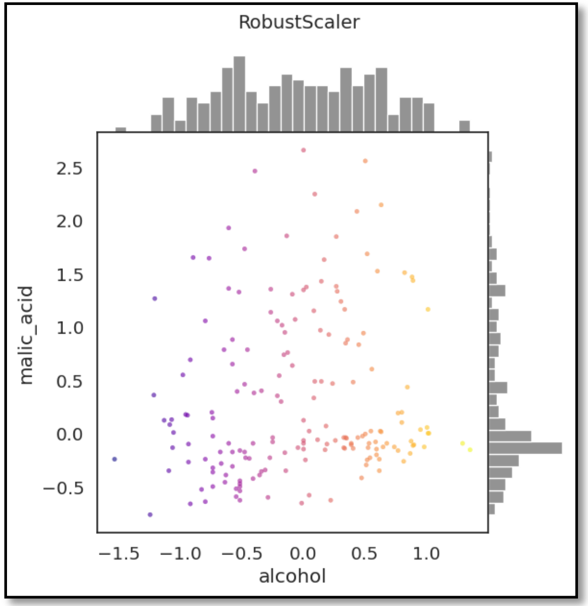




Scalers, transformers and normalizers

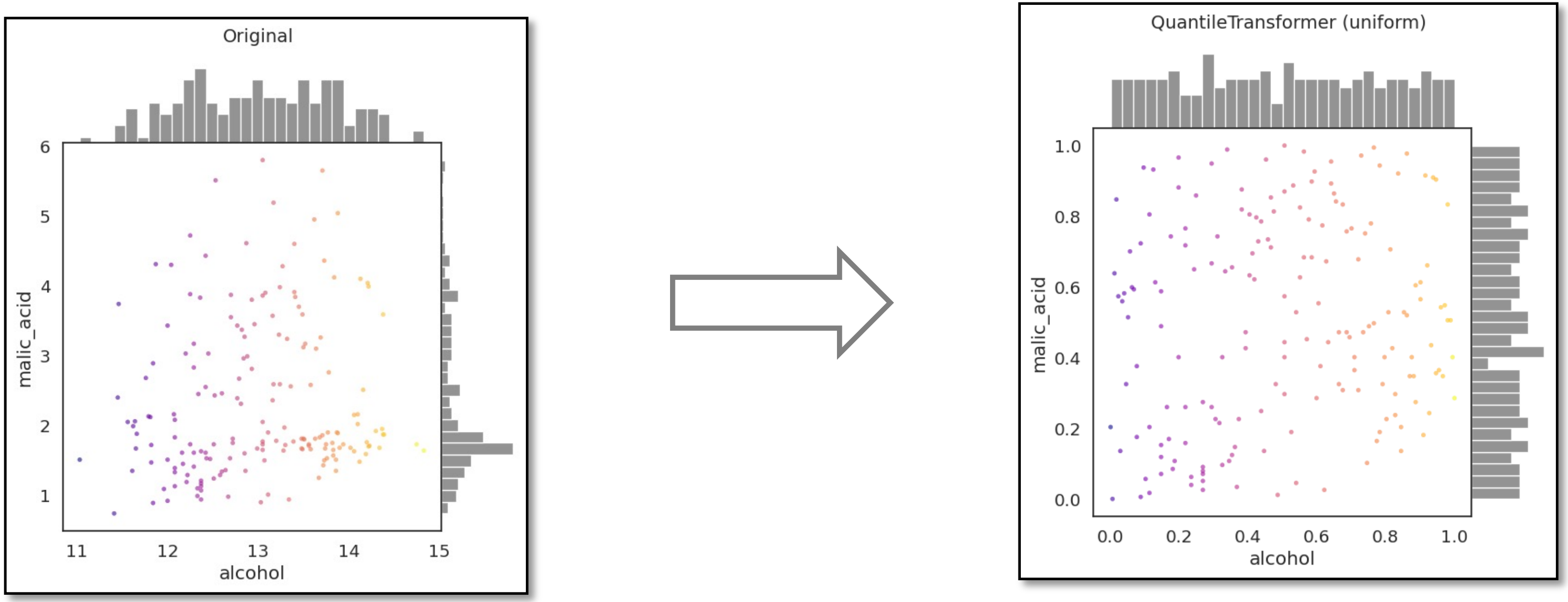
- Different variables have **different scales** and may contain **outliers**
- Difficulties when visualizing the data and, more importantly, can degrade the predictive performance of many ML algorithms (these might implicitly assume that features lie close to zero or that all features vary on comparable scales).
- The **RobustScaler** transforms each feature by subtracting its median and **scaling it by the interquartile range**.





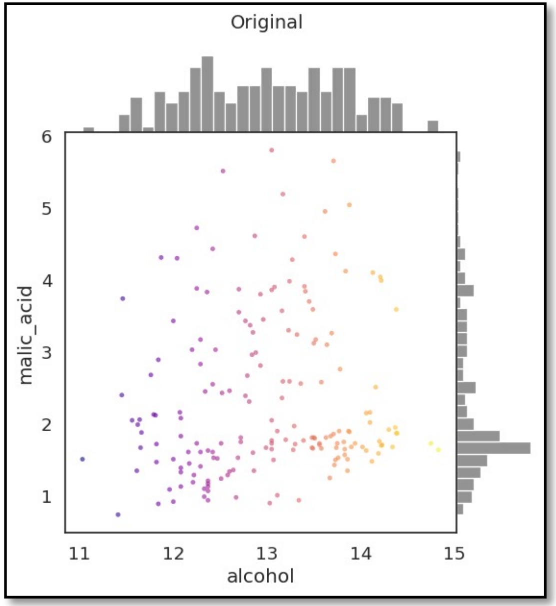
Scalers, transformers and normalizers

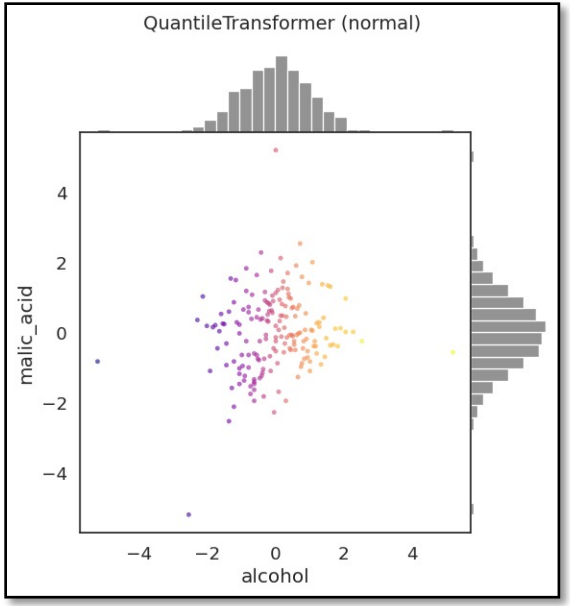
- Different variables have **different scales** and may contain **outliers**
- Difficulties when visualizing the data and, more importantly, can degrade the predictive performance of many ML algorithms (these might implicitly assume that features lie close to zero or that all features vary on comparable scales).
- The **uniform QuantileTransformer** is a nonlinear transformation that maps each feature to a **uniform** distribution



Scalers, transformers and normalizers

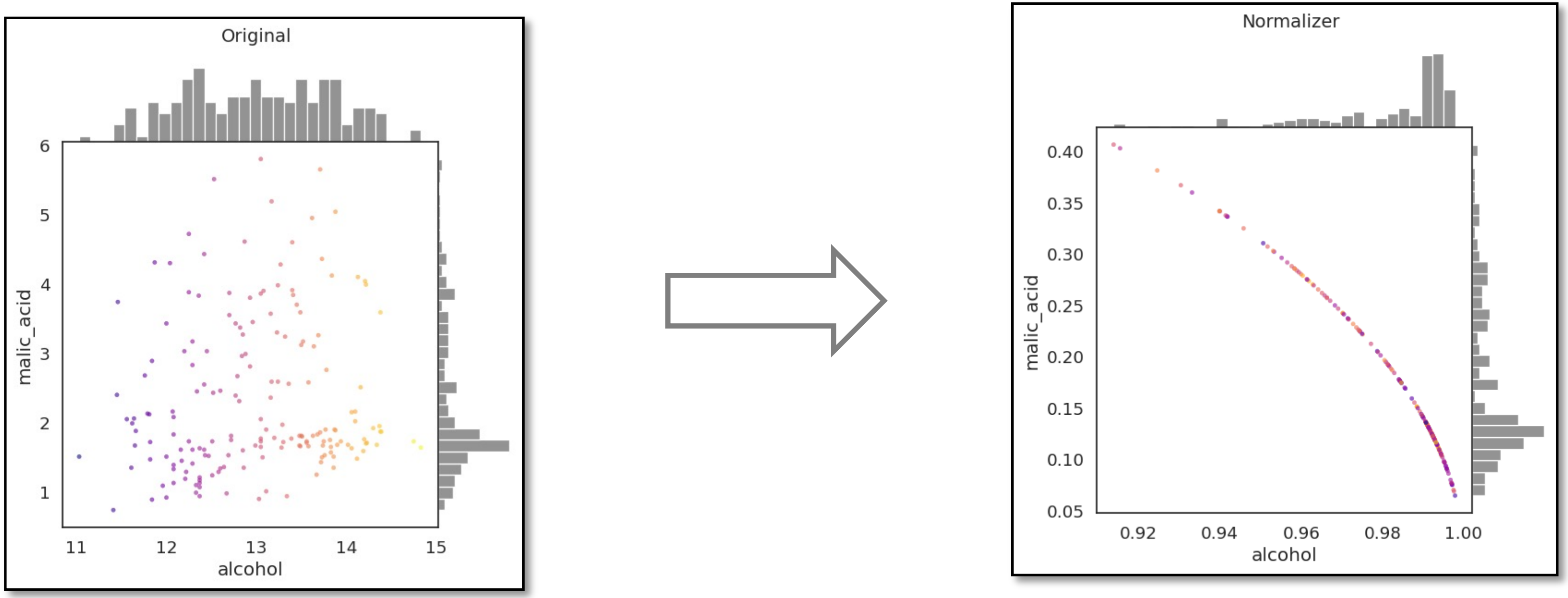
- Different variables have **different scales** and may contain **outliers**
- Difficulties when visualizing the data and, more importantly, can degrade the predictive performance of many ML algorithms (these might implicitly assume that features lie close to zero or that all features vary on comparable scales).
- The **normal QuantileTransformer** is a nonlinear transformation that forces a **Gaussian-like** distribution





Scalers, transformers and normalizers

- Different variables have **different scales** and may contain **outliers**
- Difficulties when visualizing the data and, more importantly, can degrade the predictive performance of many ML algorithms (these might implicitly assume that features lie close to zero or that all features vary on comparable scales).
- The **Normalizer** applies an normalization per sample, converting each observation into a **unit-length vector**.



Color palettes

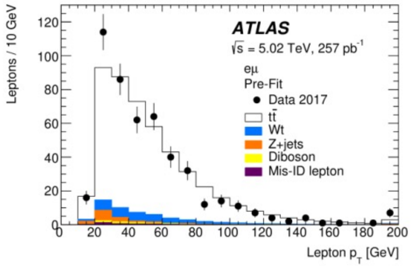
- At CERN, we make figures with several Monte Carlo predictions on them, and **color becomes essential** for their visualisation.

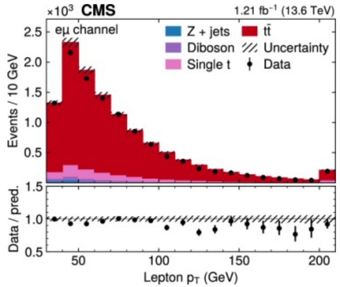
- The choice of **color palette** and plotting style can influence interpretation and usually generates substantial discussion during analysis approvals and reviews.

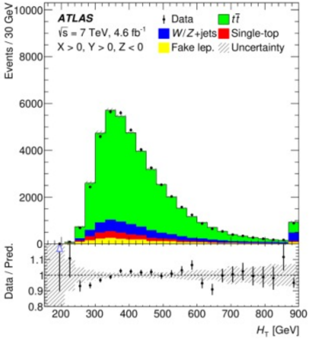
- The palette differences are also a reminder that even at the LHC, the biggest open question is not always physics… it's whether top quarks should be red or white, or green or orange. At least data is always a black point (and QCD has three colors).

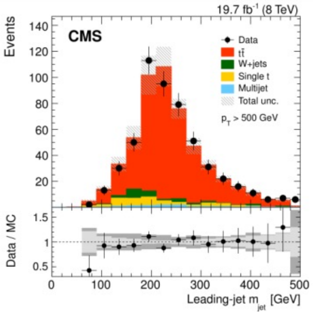
```
color='xkcd:baby poop green'
color='xkcd:baby shit brown'
color='xkcd:macaroni and cheese'
```

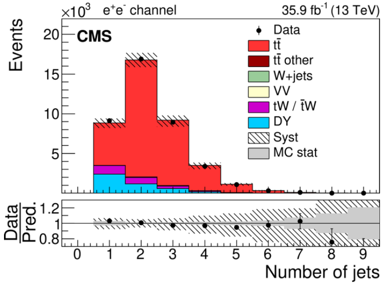
https://xkcd.com/color/rgb/

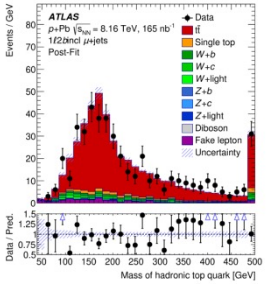






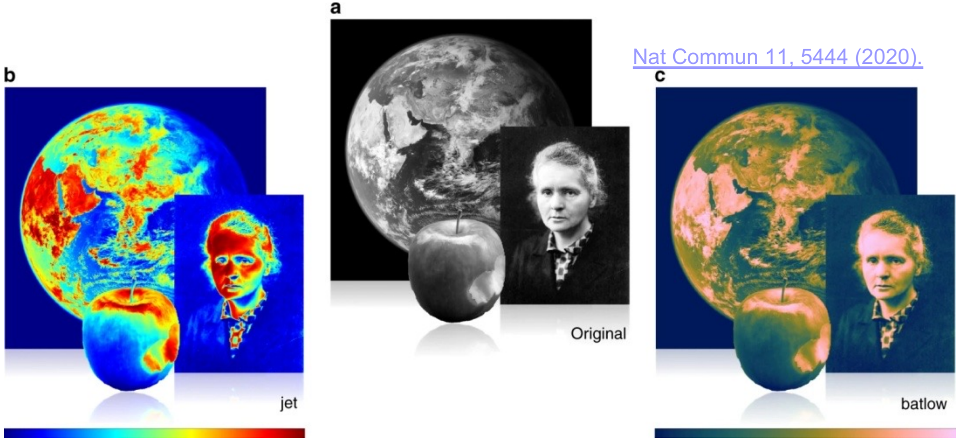




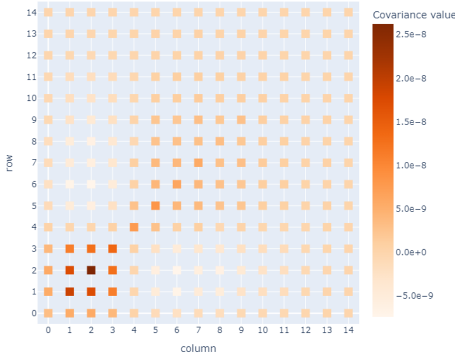


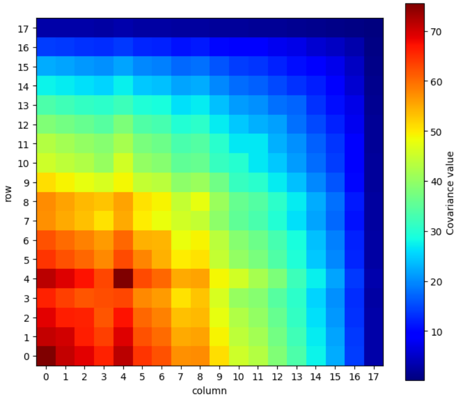
Color maps

- We also measure thousands of correlated quantities: applying a **color map** to a covariance matrix immediately **reveals structures** such as strong correlations, anti-correlations, and block-like patterns that are otherwise hidden in tables.



[Nat Commun 11, 5444 (2020).]





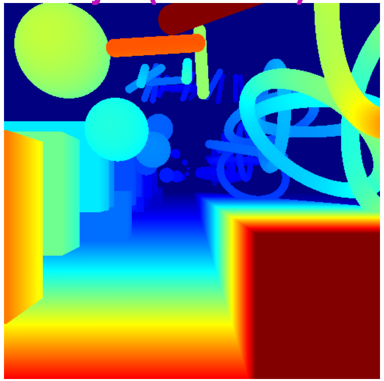
- **Notebook to try: Color.ipynb**

- Misuse of color maps in science: typical “rainbow” or “jet”-like colormaps **introduce** artificial **visual bias**, e.g. certain colors or brightness levels attract more attention, which can distort the perception of data structure

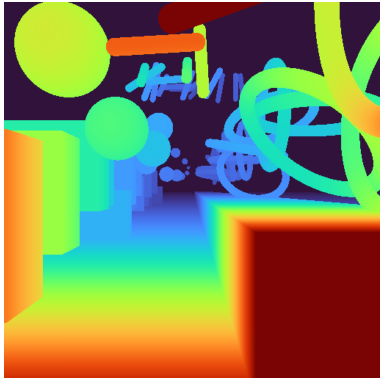
- The **distortion** can lead to **misinterpretation** and wrong conclusions!

Color maps

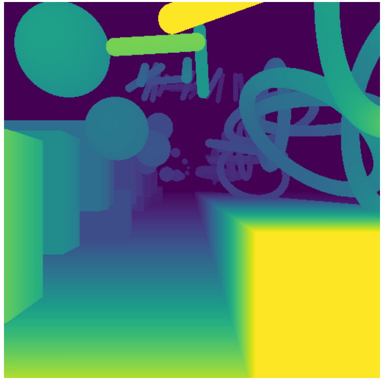
jet (rainbow)



turbo



viridis



inferno

https://research.google/blog/turbo-an-improved-rainbow-colormap-for-visualization/

- **Jet (cmap='jet‘)** is the classic rainbow map and suffers from non-uniform color and luminance jumps. These rapid changes can create **false visual boundaries**, exaggerate noise, or obscure meaningful patterns.

- **Viridis (cmap='viridis')** is a perceptually uniform, linear colormap, **widely recommended** as it avoids the major visual distortions of jet, is easy on the eyes, and preserves detail across the full range of data values.

- **Inferno (cmap='inferno')** shares Viridis linear uniform structure but has **higher contrast**.

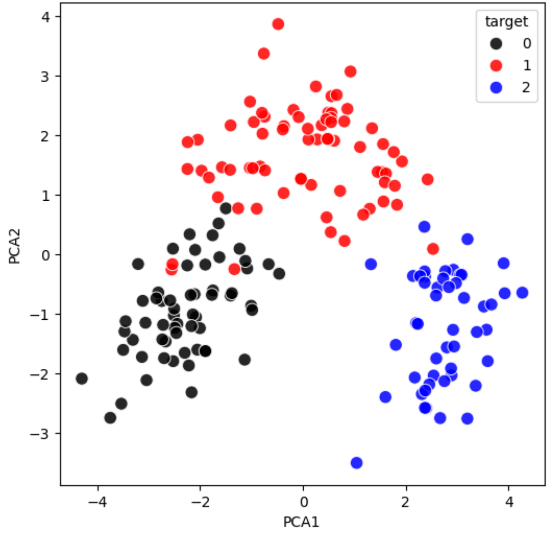
- **Turbo (cmap='turbo')** is a **redesigned rainbow** colormap with smoother gradients and controlled luminance.

Dimensionality reduction

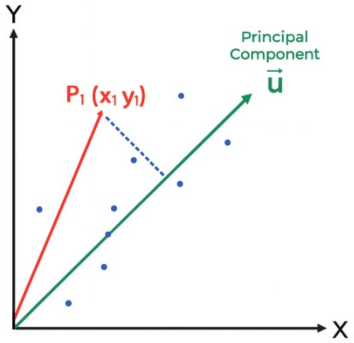
- As the **number of variables** or dimensions in a dataset **increases**, the amount of data required to obtain statistically meaningful results grows exponentially.

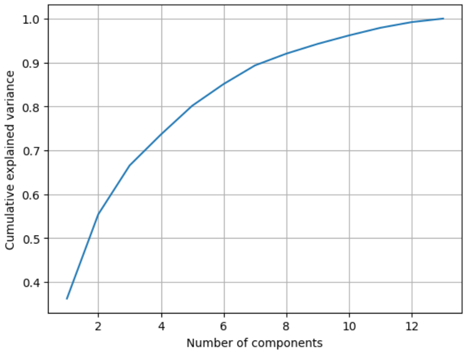
- This can lead to problems such as overfitting, increased computation time, and reduced accuracy of machine-learning models. This phenomenon is known as the **curse of dimensionality**.

- The **Principal Component Analysis** (PCA), introduced by Karl Pearson in 1901, maps the data from a higher-dimensional space to a lower-dimensional one; this is achived by maximizing the variance of the data in the lower-dimensional space.



- **Notebook to try: Dimensionality.ipynb**

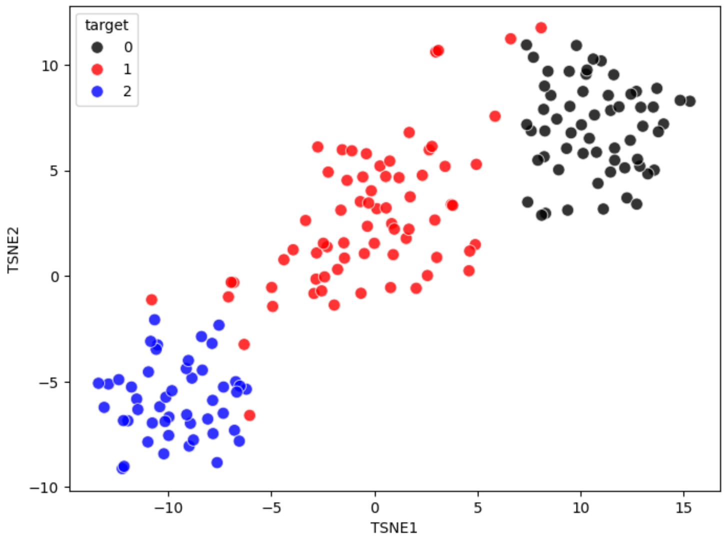


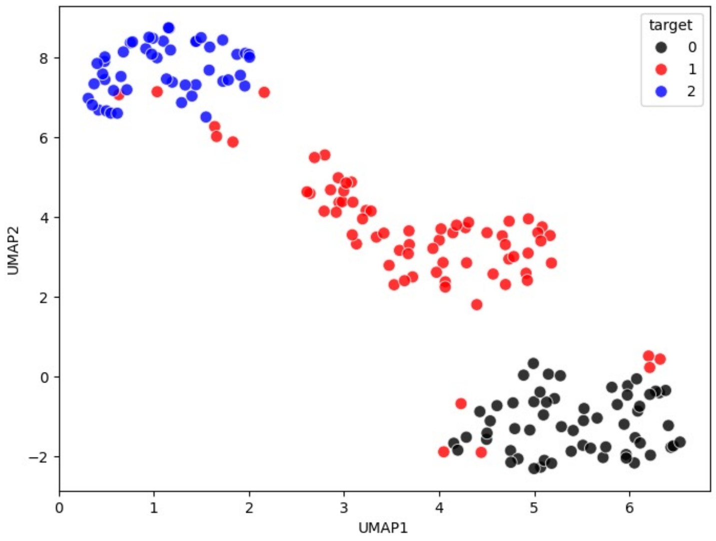


Dimensionality reduction

• While **PCA** is a **linear** dimensionality reduction technique that works best when the data lie approximately on a linear subspace, **t-SNE** (t-distributed Stochastic Neighbor Embedding) is an unsupervised **non-linear** method designed to preserve local pairwise similarities.

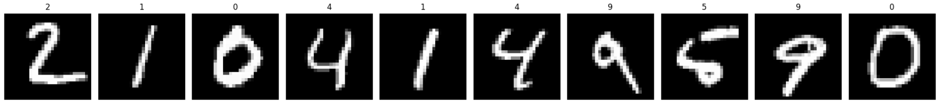
• **UMAP** (Uniform Manifold Approximation and Projection) is another **non-linear** technique based on manifold learning and topology, preserving both local neighborhoods and aspects of the global structure of the data

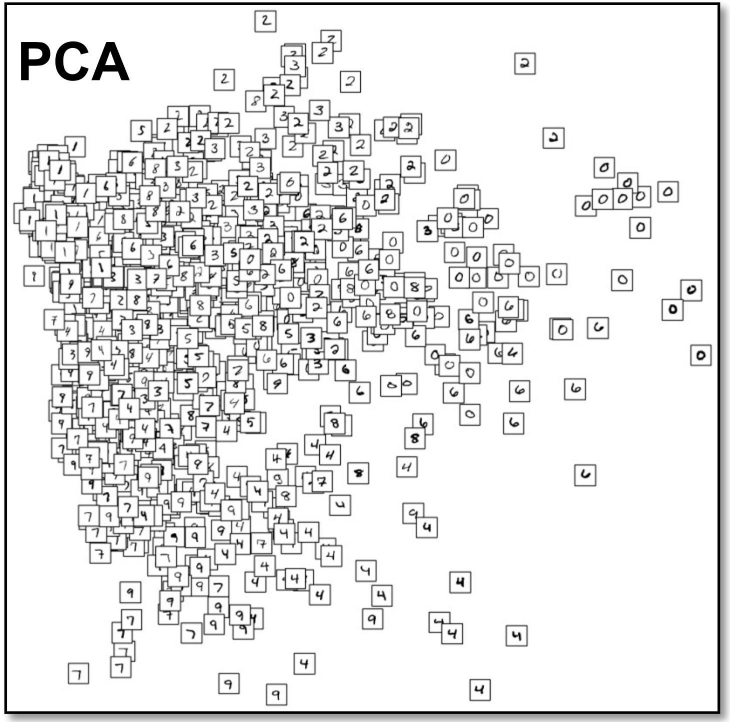


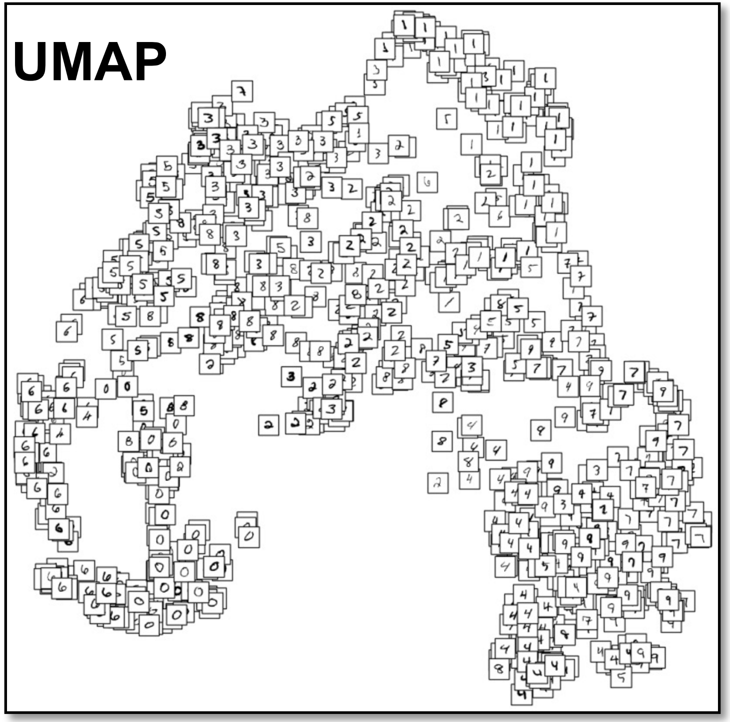


Dimensionality reduction

- Now let's use the **MNIST** dataset (handwritten digits from 0 to 9 stored as a 28×28 grayscale images) and apply PCA and UMAP to **compress** high-dimensional pixel data into a **2D visualization** that reveals digit structure.

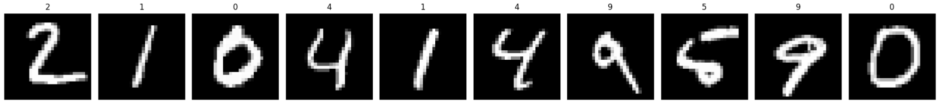


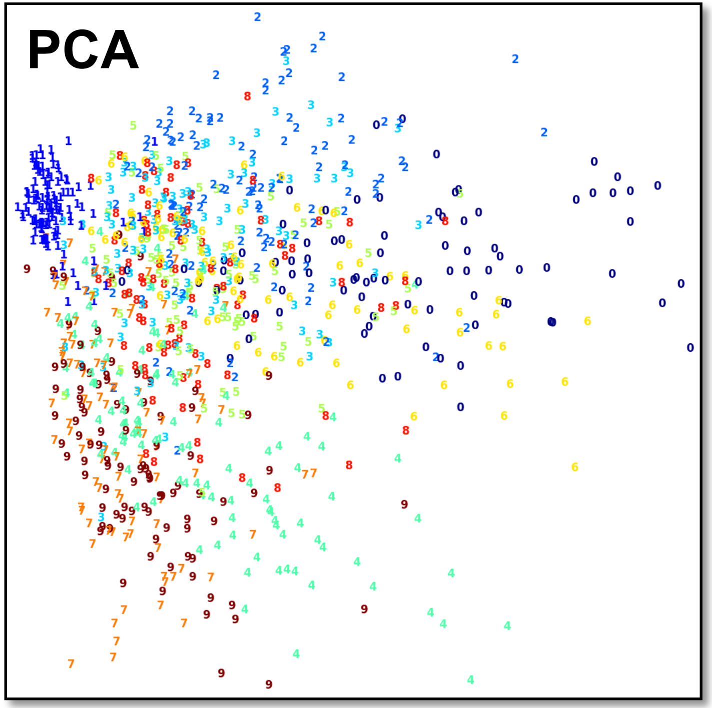


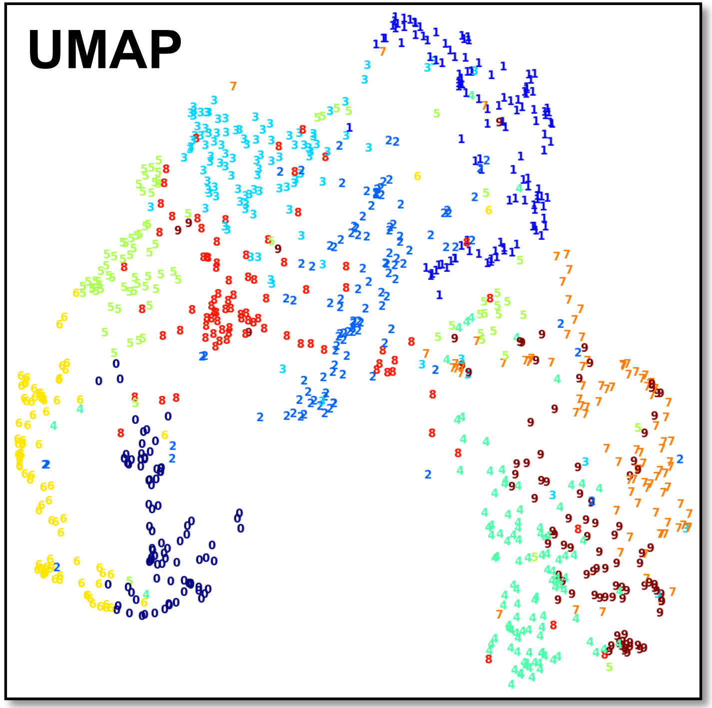


Dimensionality reduction

- Now let's use the **MNIST** dataset (handwritten digits from 0 to 9 stored as a 28×28 grayscale images) and apply PCA and UMAP to **compress** high-dimensional pixel data into a **2D visualization** that reveals digit structure.







Missing data

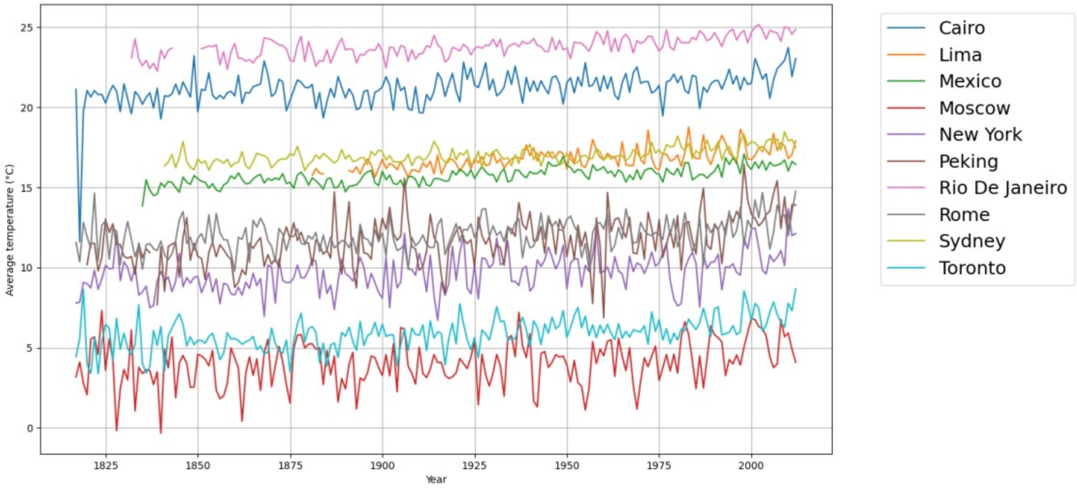
- **Global Land Temperature** (GLT) dataset collected from stations around the world, with data from 1820 and 6 features: date, average temp., city, country, latitude and longitude

- **Notebook to try: GLT_10cities.ipynb**

- Look at the "AverageTemperature" attribute it contains many **missing values**!

- Using **.isna()** we can detect the missing entries

- Always **investigate why** values are missing!




```
Percentage of missing values: 9.41%
```

- Missing values are bad because they **distort statistical analysis**, break models and calculations, ruin visualizations, and **reduce the reliability** of your conclusions.

- We have a 9.41% of missing data – this is high enough to **impact** your plots and averages if you leave it!

Missing data

- Let's **fill any missing value** using the **arithmetic mean between the closest preceding** measurement and the **closest succeeding** measurement in time, taken in the same city.

- In the case a missing value has no preceding (or succeeding) measurement, consider that the missing value is preceded (or followed) **by a 0**, and then compute the mean accordingly.

```
original = [ '', 5, '', 8, '' ]
step_1   = [ 2.5, 5, '', 8, '' ]     # (0 + 5) / 2
step_2   = [ 2.5, 5, 6.5, 8, '' ]    # (5 + 8) / 2
step_3   = [ 2.5, 5, 6.5, 8, 4 ]     # (8 + 0) / 2
```

- Afterwards, you should get something like:

City	**Date**	**Before**	**After**
Cairo	1817-01-01	NaN	20.1590
Toronto	1817-11-01	NaN	-1.2510
Rome	1818-07-01	NaN	19.4435
Rio De Janeiro	1832-05-01	NaN	23.2830
New York	1817-04-01	NaN	6.7135

- In summary, there are many ways to **handle the missing values** in your datasets:

1) **Remove** a row from the dataset if it contains a missing value - can be adopted when the **dataset is large** and the loss of information **does not affect** the overall distribution.

2) If the data has **no specific order**, missing values can be **replaced** with the **mean** (symmetric distributions) or **median** (in case of skewed data) of the corresponding attribute.

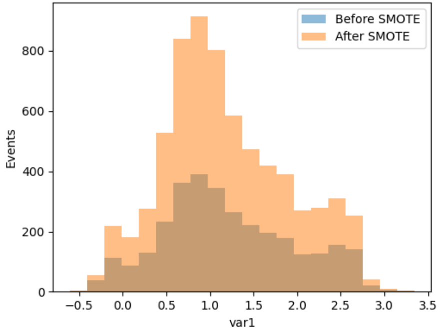
3) For **time-series data**, temporal order matters, hence missing values can be replaced using: linear **interpolation**, forward fill (copy previous value) or **rolling average** (average of previous days).

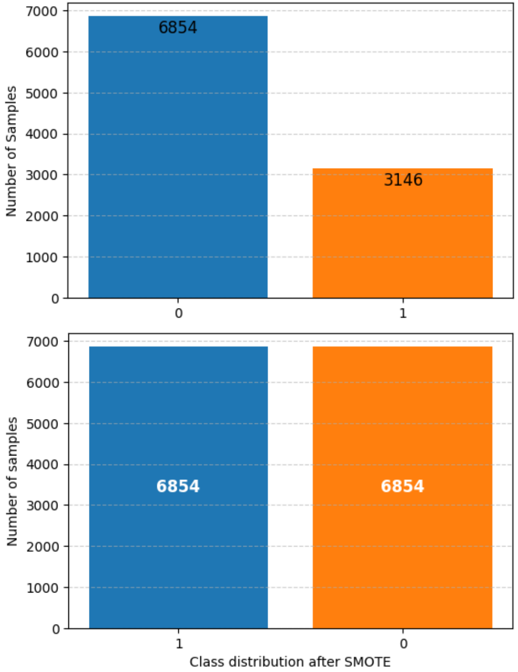
Imbalanced data

- **What can you do when your target is imbalanced?**

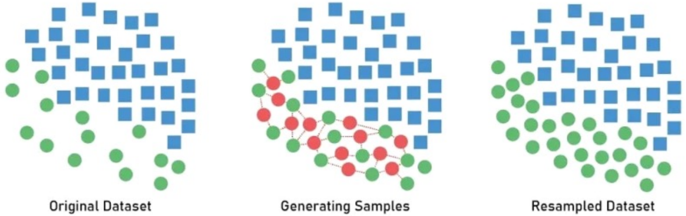
  a) Stratified sampling: keep the same class proportions in train and test sets
  b) Oversampling the minority class: generate minority samples
  c) Undersampling the majority class: randomly remove majority samples
  d) Class weighting: “balanced” in SVM or DTs; adjust loss functions in NNs
  e) The real fix: collect more data!

- **Notebook to try: [SMOTE.ipynb]()**





- **SMOTE** (Synthetic Minority Oversampling Technique) is a data-augmentation technique for tabular data:
  - ✓ finds its k-NN within the minority class using a distance metric
  - ✓ Places a new point somewhere on the line between the neighbors
  - ✓ Generates synthetic data so the class proportion is equal



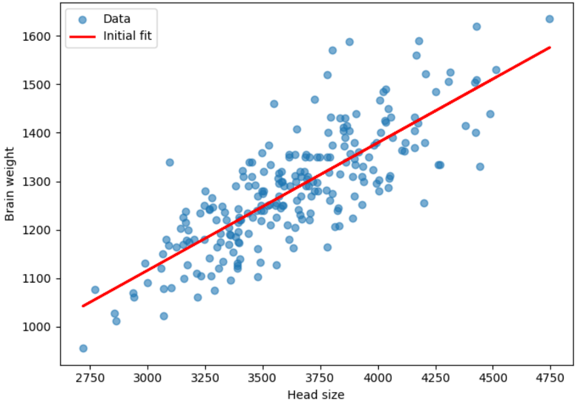
Bootstrapping

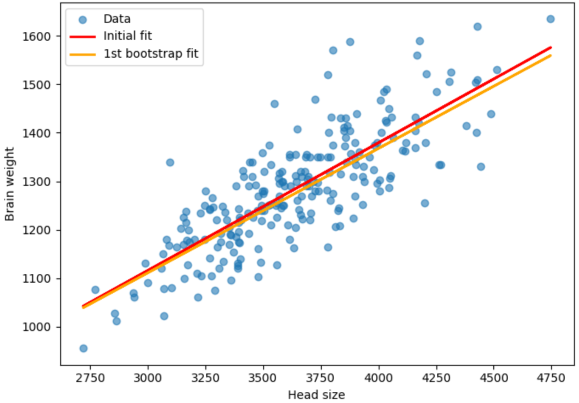
- **Bootstrapping** is a resampling method (with replacement) that **estimates statistical uncertainty** by generating many pseudo-datasets and recomputing the estimator, yielding an empirical sampling distribution.

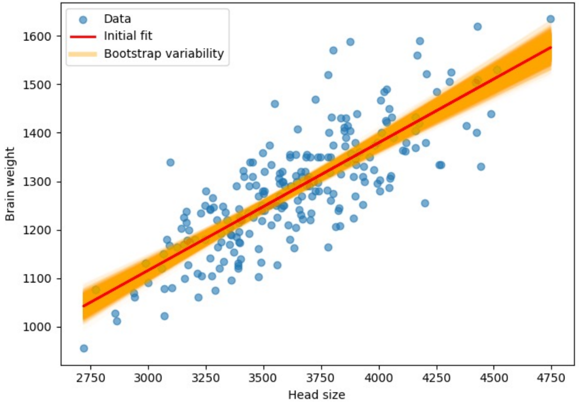
- **Notebook to try: [Bootstrap.ipynb]**

- The "**cerebro**" dataset (Gladstone, 1905) contains gender, age group, head size (cm³), and brain weight (g). We will fit a linear regression to quantify how brain weight depends on head size.

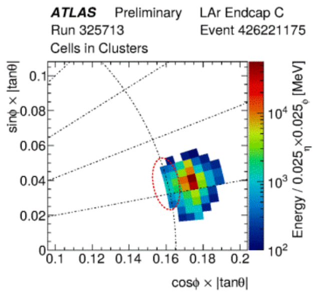
```
Original slope: 0.26
Bootstrap 95% CI: [0.24, 0.29]
```



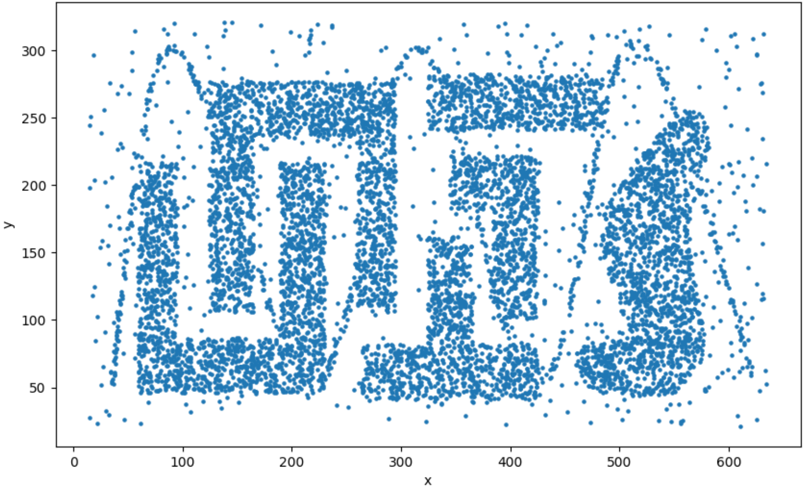


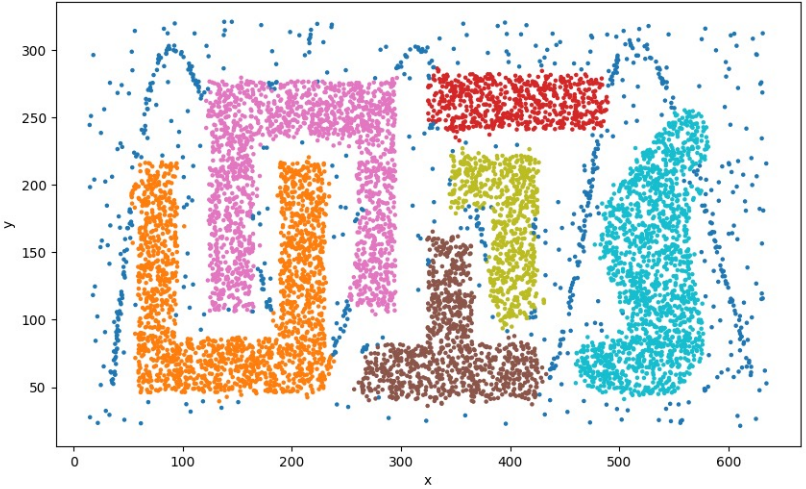


Cluster visualization



- Many tasks in HEP rely on **identifying patterns or clusters**: jets, tracks, particle-flow objects.
- **Notebook to try: [Cluster.ipynb]**
- **Chameleon** benchmark dataset: six irregular clusters + noise.
- Using it as an analogue to calorimeter data, where true energy deposits form dense clusters amid detector noise. In both cases, meaningful structure is revealed by **grouping** dense regions and **suppressing** sparse fluctuations.



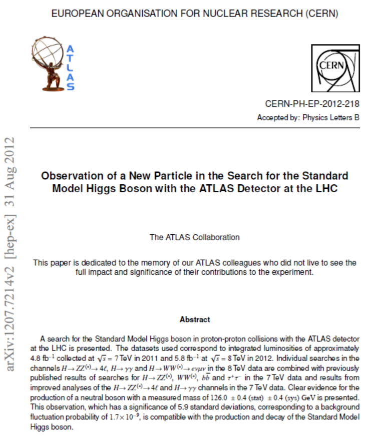


Textual data visualization

- **Word clouds**: because sometimes your supervisor wants a summary of 200 pages by tomorrow, and this is faster than learning to read at relativistic speeds.

- The **size of a word** in a word cloud is directly proportional to how many times that word appears in your processed text.

- **Notebook to try:** [Wordcloud.ipynb]()



EUROPEAN ORGANISATION FOR NUCLEAR RESEARCH (CERN)

CERN-PH-EP-2012-218
Accepted by: Physics Letters B

arXiv:1207.7214v2 [hep-ex] 31 Aug 2012

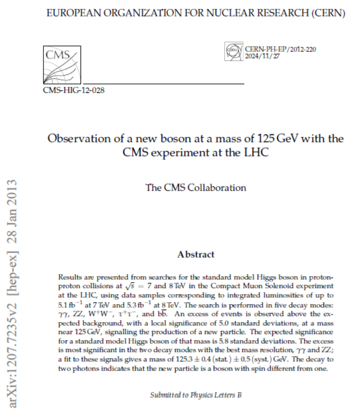
Observation of a New Particle in the Search for the Standard Model Higgs Boson with the ATLAS Detector at the LHC

The ATLAS Collaboration

This paper is dedicated to the memory of our ATLAS colleagues who did not live to see the full impact and significance of their contributions to the experiment.

Abstract

A search for the Standard Model Higgs boson in proton-proton collisions with the ATLAS detector at the LHC is presented. The datasets used correspond to integrated luminosities of approximately 4.8 fb^{-1} collected at $\sqrt{s} = 7$ TeV in 2011 and 5.8 fb^{-1} at $\sqrt{s} = 8$ TeV in 2012. Individual searches in the channels $H \to ZZ^{(*)} \to 4\ell$, $H \to \gamma\gamma$ and $H \to WW^{(*)} \to e\nu\mu\nu$ in the 8 TeV data are combined with previously published results of searches for $H \to ZZ^{(*)}$, $WW^{(*)}$, $b\bar{b}$ and $\tau^+\tau^-$ in the 7 TeV data and results from improved analyses of the $H \to ZZ^{(*)} \to 4\ell$ and $H \to \gamma\gamma$ channels in the 7 TeV data. Clear evidence for the production of a neutral boson with a measured mass of 126.0 ± 0.4 (stat) ± 0.4 (sys) GeV is presented. This observation, which has a significance of 5.9 standard deviations, corresponding to a background fluctuation probability of 1.7×10^{-9}, is compatible with the production and decay of the Standard Model Higgs boson.



EUROPEAN ORGANIZATION FOR NUCLEAR RESEARCH (CERN)

CMS-HIG-12-028

CERN-PH-EP/2012-220
2024/11/27

arXiv:1207.7235v2 [hep-ex] 28 Jan 2013

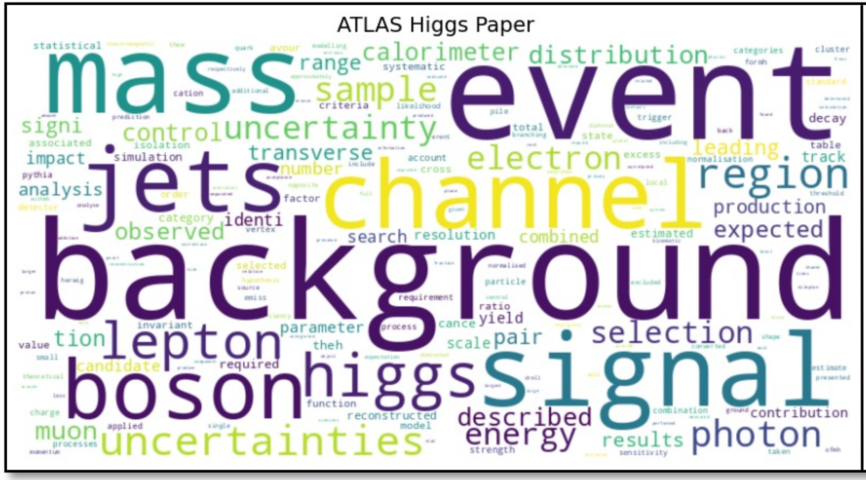
Observation of a new boson at a mass of 125 GeV with the CMS experiment at the LHC

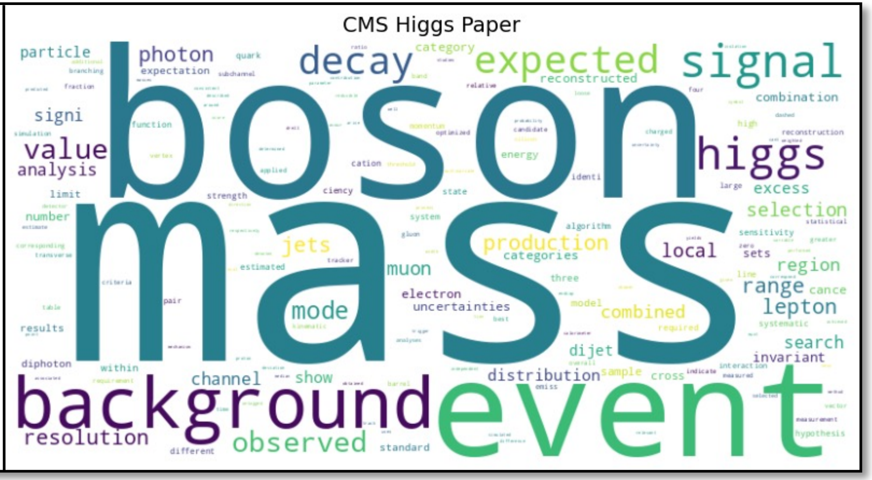
The CMS Collaboration

Abstract

Results are presented from searches for the standard model Higgs boson in proton-proton collisions at $\sqrt{s} = 7$ and 8 TeV in the Compact Muon Solenoid experiment at the LHC, using data samples corresponding to integrated luminosities of up to 5.1 fb^{-1} at 7 TeV and 5.3 fb^{-1} at 8 TeV. The search is performed in five decay modes: $\gamma\gamma$, ZZ, W^+W^-, $\tau^+\tau^-$, and $b\bar{b}$. An excess of events is observed above the expected background, with a local significance of 5.0 standard deviations, at a mass near 125 GeV, signalling the production of a new particle. The expected significance for a standard model Higgs boson of that mass is 5.8 standard deviations. The excess is most significant in the two decay modes with the best mass resolution, $\gamma\gamma$ and ZZ; a fit to these signals gives a mass of 125.3 ± 0.4 (stat.) ± 0.5 (syst.) GeV. The decay to two photons indicates that the new particle is a boson with spin different from one.

Submitted to Physics Letters B





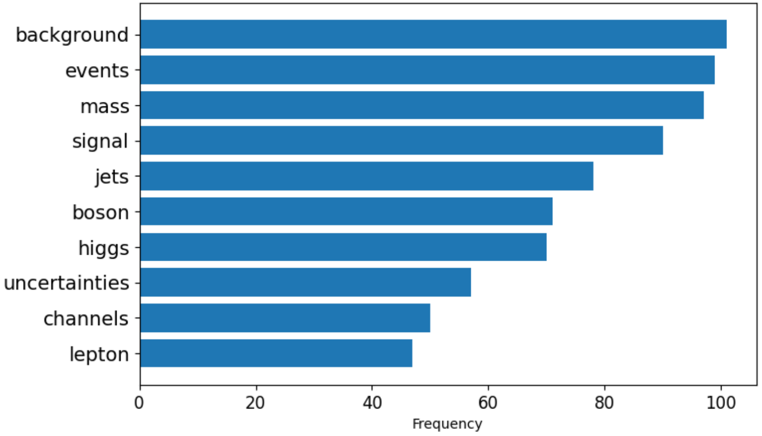
Textual data visualization

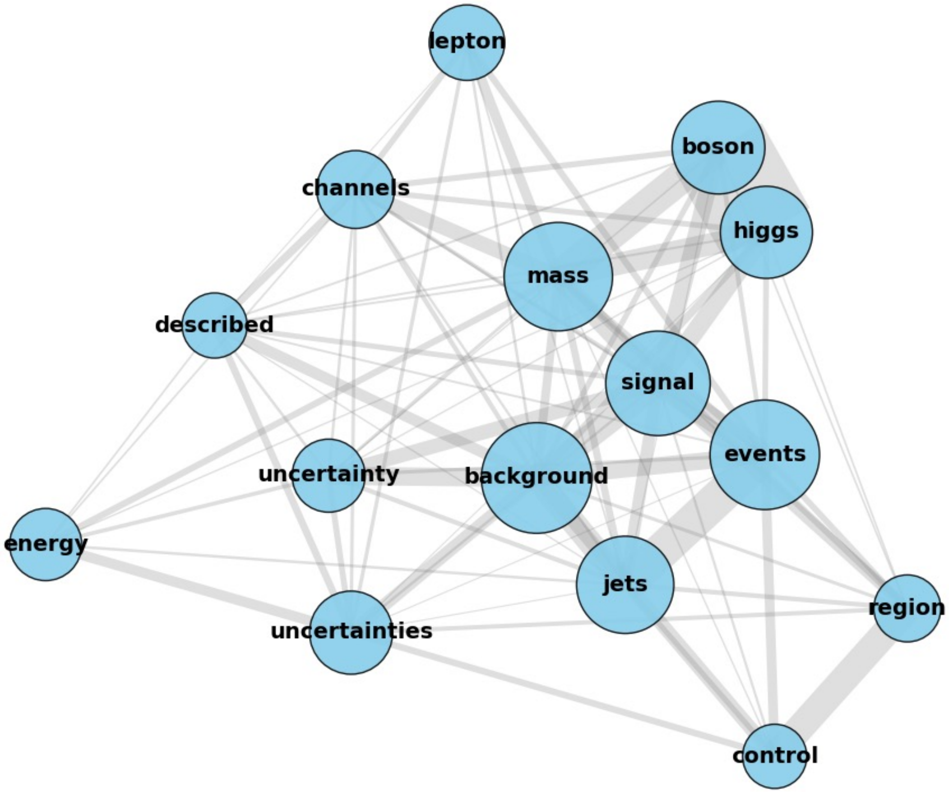
- **Word co-occurrence**: a social network of physics terms. Each circle is a frequent word in the paper

- Bigger **nodes** appear more often in the text (e.g., mass, signal, events, background).

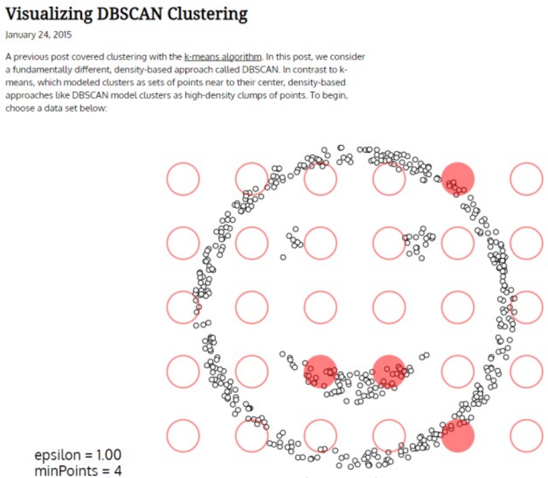
- **Lines** (edges) connect words that frequently appear near each other in the document.

- **Thicker** lines mean the words co-occur more often.

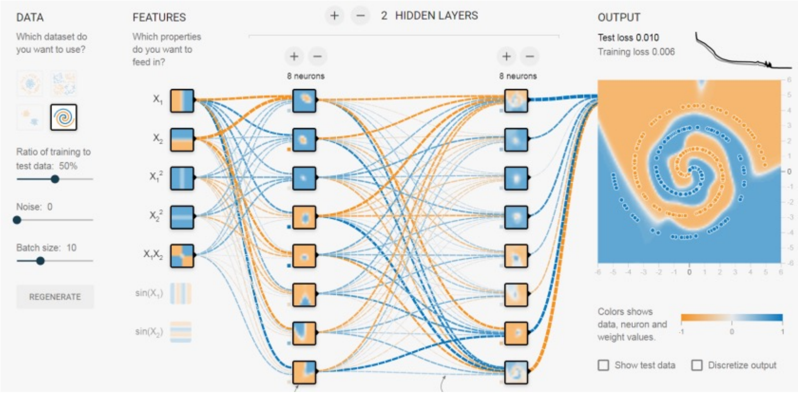




Play in your browser



https://www.naftaliharris.com/blog/visualizing-dbscan-clustering/ http://www.r2d3.us/visual-intro-to-machine-learning-part-1/



https://playground.tensorflow.org

Summary

- Data science is at the **heart of processing and interpreting** the vast amounts of data generated by the LHC.
- LHC data is analysed using sophisticated algorithms and state-of-the-art computational tools. The insights gained not only **propel advancements in particle physics** but also contribute to innovations across various scientific and engineering domains within the globe.
- **Same data science techniques** we use to extract physics from LHC, i.e. classifying events, predicting physical quantities, and discovering structure in noise, are the same tools that power **advances in other fields** worldwide.
- A variety of **statistics** can be used to summarize the empirical distribution of data-points.
- **Skewed data** distributions are common, and some summary statistics are very sensitive to outlying values.
- Data summaries always **hide some detail**, and care is required so that important information is not lost.
- **Single sets** of numbers can be visualized in charts, box-and-whisker plots and histograms.
- Consider transformations to better reveal patterns, **use the eye** to detect patterns, outliers and clusters.
- Look at **pairs of numbers** as scatter-plots, and time-series as line-graphs.
- When exploring data, a primary aim is to find factors that **explain the overall variation**.
- ***And always have fun as you never know what is hidden in your data…***

Further reading